The Most Efficient Protocol for PAKE:

What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, `xujiay@oregonstate.edu`

1 Introduction

2 Preliminaries and Technical Overview

Let n be the security parameter; we assume that all algorithms take 1^n as an input, and do not explicitly write it.

2.1 KEM Properties Used in the "Hash Everything" Version

Pseudorandom public keys. This property roughly says that an honestly generated public key is indistinguishable from uniformly random.

Definition 1. A KEM scheme has *pseudorandom public keys (UNI-PK)* if for any PPT adversary $\mathcal{A}$, $\Pr[b^* = 1]$ in the following two games are negligibly close:[1]

challenge bit $b = 0$:	challenge bit $b = 1$:
$(pk, \star) \leftarrow \mathsf{Gen}$	$pk' \leftarrow \mathcal{PK}$
$b^* \leftarrow \mathcal{A}(pk)$	$b^* \leftarrow \mathcal{A}(pk')$

This property is the same as what's called UNI-PK in [ABJ25, Definition 5]. It is needed because otherwise — as an extreme example, suppose Gen is so biased that half of the elements in $\mathcal{PK}$ (call the set of them U) are unreachable by Gen — the OEKE adversary $\mathcal{A}$ can run an offline dictionary attack as follows: $\mathcal{A}$ tries to decrypt $\mathsf{2F}^{-1}(\mathsf{pw}^*, (s, T))$ using various pw^* values, and if the result is in U then $\mathcal{A}$ knows that pw^* is not the real password pw. In this way $\mathcal{A}$ can rule out half of all candidate passwords in expectation.

[1] The challenge bit b is not actually used in the definition, but it will come in handy in the later security proofs (we sometimes need concise names for these security games).

One-wayness and a variant. This property roughly says that given a public key and a ciphertext, an adversary cannot feasibly find the key encapsulated in the ciphertext.

Definition 2. A KEM scheme is *one-way (OW)* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible:

$$\boxed{\begin{array}{l} (pk, sk) \leftarrow \mathsf{Gen} \\ (c, K) \leftarrow \mathsf{Encap}(pk) \\ K^* \leftarrow \mathcal{A}(pk, c) \\ \mathcal{A} \text{ wins if } K^* = K \end{array}}$$

We also need a stronger variant in which the adversary is additionally given access to a *plaintext-checking oracle*, which tells the adversary whether a ciphertext decapsulates to a certain key.

Definition 3. The definition of *one-wayness under one plaintext-checking attack (OW-1PCA)* is the same as that of OW, except that the adversary $\mathcal{A}$ is additionally given access to an oracle $\mathsf{PC}(sk, \star, \star)$, which on input $(c' \neq c, K')$ outputs 1 if $K' = \mathsf{Decap}(sk, c')$ and 0 otherwise. $\mathcal{A}$ is allowed to query this oracle only once.

This property is the same as what's called OW-PCA in [ABJ25, Definition 3], except that in our notion (1) the adversary can query its plaintext-checking oracle only once, and (2) the adversary is prohibited from querying the challenge ciphertext to its plaintext-checking oracle. The plaintext-checking oracle was first studied in [ABP15] in the context of public-key encryption schemes.

OW-1PCA is needed in the scenario where the OEKE adversary $\mathcal{A}$ forwards the P-to-P' message (s, T) without modification, but then modifies the P'-to-P message (c, τ') to some other (c^*, τ^*). Consider the time when P' outputs its session key SK': at this point $\mathcal{A}$ has not done any attack yet, so in the ideal world the PAKE functionality will set SK' at random. This means that $\mathcal{A}$ must not be able to compute K' (otherwise it can query $H_{\mathsf{key}}(K')$ and distinguish SK' from random); furthermore, forward secrecy requires that $\mathcal{A}$ should not compute K' even if it learns pw' later (which allows $\mathcal{A}$ to compute $pk' := \mathsf{2F}^{-1}(\mathsf{pw}', (s, T))$) — which is why we give the public key to the adversary in the security game.

Furthermore, later when $\mathcal{A}$ modifies (c, τ') to $(c^* \neq c, \tau^* = H_{\mathsf{tag}}(\ldots, K^*))$ for K^* of its choice, it can check if $K^* = \mathsf{Decap}(sk, c^*)$ by observing if P's session key $SK = H_{\mathsf{key}}(K^*)$. This corresponds to the (single) query to the plaintext-checking oracle. (This requires that P and P''s passwords match; otherwise sk on the P side does not correspond to pk' on the P' side.)

Anonymity and a variant. This property roughly says that given a public key and a ciphertext, an adversary cannot feasibly tell whether the ciphertext corresponds to this public key or some other (freshly sampled) public key.

Definition 4. A KEM scheme is *anonymous (ANO)* if for any PPT adversary $\mathcal{A}$, $\Pr[b^* = 1]$ in the following two games are negligibly close:

challenge bit $b = 0$:	challenge bit $b = 1$:
$(pk, sk) \leftarrow \mathsf{Gen}$	$(pk, sk) \leftarrow \mathsf{Gen}$
$(c, \star) \leftarrow \mathsf{Encap}(pk)$	$(pk', \star) \leftarrow \mathsf{Gen}$
$b^* \leftarrow \mathcal{A}(pk, c)$	$(c, \star) \leftarrow \mathsf{Encap}(pk')$
	$b^* \leftarrow \mathcal{A}(pk, c)$

Similar to OW-1PCA, we also need the 1PCA variant of ANO.

Definition 5. The definition of *anonymity under one plaintext-checking attack (ANO-1PCA)* is the same as that of ANO, except that the adversary $\mathcal{A}$ is additionally given access to an oracle $\mathsf{PC}(sk, \star, \star)$, which on input $(c' \neq c, K')$ outputs 1 if $K' = \mathsf{Decap}(sk, c')$ and 0 otherwise. $\mathcal{A}$ is allowed to query this oracle only once.

This property is the same as what's called ANO-PCA in [ABJ25, Definition 4], except that in our notion (1) the adversary can query its plaintext-checking oracle only once, and (2) the adversary is not given pk' (whereas in their notion pk' is generated and given to the adversary in both games).

ANO-1PCA is needed again in the scenario where the OEKE adversary $\mathcal{A}$ forwards the P-to-P' message (s, T) without modification, but then modifies the P'-to-P message (c, τ'). Consider c: in the real world c is the result of $\mathsf{Encap}(pk)$ (assuming the two parties' passwords match), whereas in the ideal world the simulator does not know pk at this point (it learns no information about pw, so it cannot decrypt (s, T) and learn pk) and has to simulate c as the result of $\mathsf{Encap}(pk')$ for a fresh pk'. $\mathcal{A}$ should not be able to tell these two cases apart (even after it learns pw later and uses it to compute $pk := \mathsf{2F}^{-1}(\mathsf{pw}, (s, T))$), which is ANO. 1PCA is needed for the same reason as in OW-1PCA.

Anonymity with uniform public keys and random oracles. Finally, we define a highly tailored KEM property that is implied by the preceding more standard properties. This property is for facilitating the security proof of OEKE only; we do not expect it to be of independent interest.

Definition 6. A KEM scheme is *anonymous with q uniform public keys and random oracles (q-UNI-ANO-RO)* if $\Pr[b^* = 1]$ in the following two games are negligibly close:

challenge bit $b = 0$:	challenge bit $b = 1$:
$pk_1, \ldots, pk_q \leftarrow \mathcal{PK}$	$pk', pk_1, \ldots, pk_q \leftarrow \mathcal{PK}$
$i \leftarrow \mathcal{A}(pk_1, \ldots, pk_q)$	$i \leftarrow \mathcal{A}(pk_1, \ldots, pk_q)$
$(c, K) \leftarrow \mathsf{Encap}(pk_i)$	$(c, \star) \leftarrow \mathsf{Encap}(pk')$
$h_1 := H_1(K); h_2 := H_2(K)$	$h_1 \leftarrow \{0,1\}^n; h_2 \leftarrow \{0,1\}^{2n}$
$b^* \leftarrow \mathcal{A}(c, h_1, h_2)$	$b^* \leftarrow \mathcal{A}(c, h_1, h_2)$

Here, H_1 and H_2 are ROs with range $\{0,1\}^n$ and $\{0,1\}^{2n}$, respectively.

This property emerges when the OEKE adversary $\mathcal{A}$ sends (s^*, T^*) to P' which contains a wrong password guess. In the real world, (s^*, T^*) decrypt to a uniformly random KEM public key, which is then used to compute c, SK, τ (the $b = 0$ game). In simulation, we want to remove the decryption process and outright use a freshly sampled public key to compute c, SK, τ (the $b = 1$ game). (h_1, h_2 in Definition 6 correspond to SK, τ in the protocol, respectively.) What complicates things is that $\mathcal{A}$ can decrypt a large number of (s^*, T^*) pairs *under both correct and incorrect passwords*, resulting in a large number of pk^* values, *before* sending (s^*, T^*) to P' (or even before initiating P''s session); at this time the reduction does not know which (s^*, T^*) will be used by $\mathcal{A}$, or which password is correct, yet it has to program the public key (the "right" one that will be used in P''s algorithm) into the decryption process. As such, we need to give the reduction polynomially many public keys.

It is not hard to see that q-UNI-ANO-RO is implied by the more standard properties:

Lemma 1. *Suppose the UNI-PK, OW and ANO properties hold for a KEM scheme. Then for any polynomial q, the q-UNI-ANO-RO property also holds for the KEM scheme.*

The proof is deferred to Section C.

2.2 Additional KEM Properties Used in Other Versions

Weak non-malleability. This property roughly says that given an honest tuple of (pk, K, c), the adversary cannot feasibly find another valid (pk, K, c^*) tuple (i.e., a new ciphertext that works for the same public key and the same KEM key). This essentially requires that the adversary cannot merely change the format of the ciphertext without changing its actual content, which is a very weak property (in fact it is quite a stretch to even call it "non-malleability").

Definition 7. A KEM scheme is *weakly non-malleable (WNM)* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible:

$$\boxed{\begin{array}{l} (pk, sk) \leftarrow \mathsf{Gen} \\ (c, K) \leftarrow \mathsf{Encap}(pk) \\ c^* \leftarrow \mathcal{A}(pk, c) \\ \text{abort if } c^* = c \\ \mathcal{A} \text{ wins if } \mathsf{Decap}(sk, c^*) = K \end{array}}$$

WEM is needed in OEKE varants where c is removed from H_{tag} inputs. Then we must prevent the OEKE adversary $\mathcal{A}$ from forwarding the P-to-P' message (s, T) without modification, but then modifying the P'-to-P message (c, τ') to some (c^*, τ) which makes P's check pass.[2] (If c is included in H_{tag}

[2]It is, in fact, unclear whether this is an "actual" attack: $\mathcal{A}$ causes the two parties to output the same session key by changing c and forwarding all other protocol messages, but $\mathcal{A}$ does not learn the session key or the password. However, at least such an $\mathcal{A}$ makes the UC simulation fail, since the simulator is not able to tell that $\mathcal{A}$ is sending a valid c^* (it does not know sk). The same comment is true for the attack in the "key binding" paragraph below.

then P's check will not pass unless $(\dots, c, \dots)$ and $(\dots, c^*, \dots)$ form a collision for H_{tag}.) This should not happen even after $\mathcal{A}$ later learns pw and uses it to compute $pk := \mathsf{2F}^{-1}(\mathsf{pw}, (s, T))$.

Unpredictability. This property roughly says that an adversary $\mathcal{A}$ cannot feasibly output a ciphertext/KEM key pair (c^*, K^*) "out of thin air", such that (pk, K^*, c^*) is a valid tuple for an honestly generated public key pk that $\mathcal{A}$ does not know.

Definition 8. A KEM scheme is *unpredictable (UNPRE)* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible:

$$\boxed{\begin{array}{l} (\star, sk) \leftarrow \mathsf{Gen} \\ (c^*, K^*) \leftarrow \mathcal{A}() \\ \mathcal{A} \text{ wins if } \mathsf{Decap}(sk, c^*) = K^* \end{array}}$$

While UNPRE is a very weak property, it is not implied by other properties we have seen. For example, a (contrived) KEM scheme could have a "special" pair of ciphertext c and KEM key K such that $\mathsf{Decap}(sk, c) = K$ for any valid secret key sk; this does not necessarily violate any of the standard properties (in particular, OW is not necessarily violated since the probability that $\mathsf{Encap}(pk)$ hits c might be negligible).

UNPRE is needed in OEKE varants where pk is removed from H_{tag} inputs. If the OEKE adversary $\mathcal{A}$ can come up with c^*, K^* that break the UNPRE property, then it can compute $\tau^* := H_{\mathsf{tag}}(\dots, K^*)$ and send (c^*, τ^*) to P. This causes P's check to pass, and P will output $SK := H_{\mathsf{key}}(K^*)$, which $\mathcal{A}$ can compute. If pw is also removed from H_{tag} inputs then $\mathcal{A}$ can compute P's session key without knowing the password, which clearly breaks the security of OEKE. If pw is included in H_{tag} then it seems that $\mathcal{A}$ is not "actually" breaking the security of OEKE, but at least the UC simulator cannot tell that $\mathcal{A}$ is performing this attack (it does not know pk), so it is unclear how the simulator should proceed in this case. (If pk is included in H_{tag} then the attack above does not work, since $\mathcal{A}$ cannot compute the correct τ^* without knowing pk.)

Remark 1. [ABJ25] seems to have noticed that some form of unpredictability is needed if pk is removed from H_{tag} inputs. [ABJ25, Section 5] says:

> *We note that, although it is tempting to remove pk from the H_{tag} input by arguing that $(\mathsf{pw}, [(s, T)])$ fix a single pk following the IC intuition. However, this gives rise to cases in the proof where the adversary may try to break the protocol by delivering ciphertexts that are valid for all public keys. It is likely that these cases can be excluded by assuming additional properties on the KEM, but we choose not to do so.*

However, it is unclear to us what "ciphertexts that are valid for all public keys" exactly means.

Key binding. This property roughly says that given an honest tuple of (pk', K', c) and another honestly generated public key pk, an adversary cannot feasibly come up with a ciphertext c^* such that (pk, K', c^*) is also a valid tuple. In other words, the KEM key K' binds a public key/ciphertext pair, although we give the adversary the capability of choosing one ciphertext only, and both public keys and the other ciphertext are simply given to the adversary. (In the actual definition, we let pk' be sampled uniformly at random, rather than honestly generated.)

Definition 9. A KEM scheme is *key binding (KBIND)* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible:

$$\boxed{\begin{array}{l} (pk, sk) \leftarrow \mathsf{Gen} \\ pk' \leftarrow \mathcal{PK} \\ (c, K') \leftarrow \mathsf{Encap}(pk') \\ c^* \leftarrow \mathcal{A}(pk, pk', c, K') \\ \mathcal{A} \text{ wins if } \mathsf{Decap}(sk, c^*) = K' \end{array}}$$

KBIND is needed in OEKE varants where pw, pk and c are *all* removed from H_{tag} inputs. The attack only works in the setting where P's password pw and P''s password pw' are different. It is more subtle than previous attacks, so let us describe the OEKE adversary $\mathcal{A}$ in more detail:

1. On protocol message (s, T) from P, forward (s, T) to P' without modification. This causes P' to decrypt $pk' := 2\mathsf{F}^{-1}(\mathsf{pw}', (s, T))$, which is uniformly random and independent of pk since $\mathsf{pw}' \neq \mathsf{pw}$. P' then sends $(c, \tau' := H_{\mathsf{tag}}(s, T, K'))$ to P (intercepted by $\mathcal{A}$) and outputs $SK' := H_{\mathsf{tag}}(K')$.

2. Suppose $\mathcal{A}$ now learns pw' (i.e., after P''s session completes); then $\mathcal{A}$ can use it to recover pk'. If $\mathcal{A}$ can find another c^* that breaks the KBIND property, then $\mathcal{A}$ can send (c^*, τ') to P. P will compute the same K' upon decapsulation, so its check $\tau' \stackrel{?}{=} H_{\mathsf{tag}}(s, T, K')$ will pass and P will output the same $H_{\mathsf{tag}}(K')$ as P' does.

Similar to the attack in the "weak non-malleability" paragraph, $\mathcal{A}$ causes the two parties to output the same session key by changing c and forwarding all other protocol messages. (If pw is included in H_{tag} then P's check will not pass unless $(\mathsf{pw}, \dots)$ and $(\mathsf{pw}', \dots)$ form a collision for H_{tag}. If pk is included in H_{tag} then P's check will not pass unless either (1) the uniformly random and independent pk' happens to be equal to pk, or (2) $(\dots, pk, \dots)$ and $(\dots, pk', \dots)$ form a collision for H_{tag}. Similarly, if c is included in H_{tag} then P's check will not pass unless $(\dots, c, \dots)$ and $(\dots, c^*, \dots)$ form a collision for H_{tag}.)

Note that the OEKE adversary above only knows pk' and c, whereas in the definition of KBIND we additionally give pk and K' to the adversary. This is because the *reduction* in our OEKE security proof (see Claim 4) needs pk to simulate the P-to-P' message (s, T), and needs K' to simulate P''s output SK'. It is possible that a reduction to a weaker KBIND property (where pk, K' are not given the adversary) can go through by sampling s, T, K' at random, but that would require changing the order of the games in the security proof, and we do not explore this possibility.

Robustness. This property roughly says that given two honestly generated public keys pk, pk', an adversary cannot feasibly come up with a ciphertext c^* that decapsulates to the same KEM key K^* under the corresponding secret keys sk, sk'.

Definition 10. A KEM scheme is *robust (ROB)* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible:

$$\boxed{\begin{array}{l} (pk, sk) \leftarrow \mathsf{Gen} \\ (pk', sk') \leftarrow \mathsf{Gen} \\ (c^*, K^*) \leftarrow \mathcal{A}(pk, pk') \\ \mathcal{A} \text{ wins if } \mathsf{Decap}(sk, c^*) = \mathsf{Decap}(sk', c^*) = K^* \end{array}}$$

Robustness has been studied in the context of public-key encryption schemes [ABN10] and authenticated encryption schemes [FOR17]. In these prior works, robustness requires that the adversary cannot feasibly come up with a ciphertext that is *valid* under two public keys. Here we only need the adversary not to come up with a ciphertext that decapsulates to the *same* KEM key, which is a much weaker property.

ROB is needed in OEKE varants where pw and pk are *both* removed from H_{tag} inputs. This is to prevent the following attack: The OEKE adversary $\mathcal{A}$ completely disregards P', and on (s, T) from P, $\mathcal{A}$ chooses two password guesses $\mathsf{pw}_1^*, \mathsf{pw}_2^*$ and computes $pk_1^* := \mathsf{2F}^{-1}(\mathsf{pw}_1^*, (s, T)), pk_2^* := \mathsf{2F}^{-1}(\mathsf{pw}_2^*, (s, T))$. If $\mathcal{A}$ can find (c^*, K^*) that breaks the ROB property for pk_1^*, pk_2^*, it then sends $(c^*, \tau^* = H_{\mathsf{tag}}(s, T, c^*, K^*))$ to P. As long as one of $\mathsf{pw}_1^*, \mathsf{pw}_2^*$ is equal to P's password pw, P's check will pass and $\mathcal{A}$ can compute P's session key $SK = H_{\mathsf{key}}(K^*)$. In this way $\mathcal{A}$ can make two password guesses per session, which is not allowed by PAKE security. (If either pw or pk is included in H_{tag} then the attack above does not work, since $\mathcal{A}$ needs to commit to a single (pw^*, pk^*) pair while sending τ^*, unless $\mathcal{A}$ finds a collision in H_{tag}.)

2.3 Example: Diffie–Hellman KEM

It is instructive to see what cryptographic assumptions the various KEM properties are, in the well-known Diffie–Hellman KEM. This KEM scheme works in a finite cyclic group with (public) generator g and prime order q superpolynomial in n, and

- Gen samples $x \leftarrow \mathbb{Z}_q$, computes $X := g^x$, and outputs (X, x).
- $\mathsf{Encap}(X)$ samples $y \leftarrow \mathbb{Z}_q$, computes $Y := g^y$ and $K := X^y$, and outputs (Y, K).

- $\mathsf{Decap}(x, Y)$ outputs Y^x.

(Note that the computation of the ciphertext Y does not rely on the public key X; in terms of key exchange, Diffie–Hellman is 1-simultaneous round, whereas KEM can model any 2-message-flow key exchange protocol.) The Diffie–Hellman KEM is perfectly correct, and $p_{\text{guess}}(X) = 1/q$. Furthermore, it satisfies various properties we have defined (below we always assume $x, y \leftarrow \mathbb{Z}_q$):

- UNI-PK says that $X = g^x$ is indistinguishable from uniformly random, which holds perfectly.

- OW says that given $(X = g^x, Y = g^y)$ it is infeasible to compute $K = g^{xy}$, which is the Computational Diffie–Hellman (CDH) assumption.

- OW-1PCA says that given $(X = g^x, Y = g^y)$ and an oracle that on input $(Y' \neq Y, K')$ checks if $(Y')^x = K'$ (which the adversary can query only once), it is infeasible to compute $K = g^{xy}$. This is the Gap Diffie–Hellman (GDH) assumption, except that (1) when the adversary makes a DH oracle query, its first input is fixed to be X, (2) when the adversary makes a DH oracle query, its second input must not be equal to Y, and (3) the adversary can query the (restricted) DH oracle only once.[3] This limited version of GDH can be reduced to CDH.

- ANO says that $(X = g^x, Y = g^y)$ and $(X = g^x, Y' = g^{y'})$ for another $y' \leftarrow \mathbb{Z}_q$ are indistinguishable, which holds perfectly. (This property holds perfectly for any KEM scheme where the ciphertext does not rely on the public key.) It of course holds perfectly even if the adversary is additionally given the plaintext-checking oracle.

- WNM says that given $(X = g^x, Y = g^y)$ it is infeasible to come up with $Y^* \neq Y$ such that $(Y^*)^x = X^y$, which holds perfectly.

- UNPRE says that it is infeasible to come up with (Y^*, K^*) such that $K^* = (Y^*)^x$, which holds perfectly.

- KBIND says that given $(X = g^x, X', Y = g^y, (X')^y)$ where X' is a freshly sampled random group element, it is infeasible to come up with Y^* such that $(Y^*)^x = (X')^y$. We claim that this is implied by the CDH assumption. Since $x = 0$ with negligible probability $1/q$, below we assume this does not happen, and the adversary's task is to compute $((X')^y)^{\frac{1}{x}}$. Let $Z = (X')^y$, and consider a stronger adversary that is additionally given $\log_g Z$. Note that X and Z are just two independent and uniformly random group elements, so computing $Z^{\frac{1}{x}} = (g^{\frac{1}{x}})^{\log_g Z}$ is equivalent to computing $g^{\frac{1}{x}}$. But computing $g^{\frac{1}{x}}$ given $X = g^x$ is the inverse Diffie–Hellman problem, which is well-known to be equivalent to CDH.[4]

[3] Some people's definition of GDH (e.g., [LLMT25, Section 2]) has restriction (1) in place, namely the oracle only takes two inputs (Y', Z) and checks if $Z = (Y')^x$.

[4] Inverse Diffie–Hellman is equivalent to square Diffie–Hellman [BDZ03, Section 2.2], which in turn is equivalent to CDH [BDZ03, Section 2.1].

- ROB says that given $(X = g^x, X' = g^{x'})$ for $x' \leftarrow \mathbb{Z}_q$ independent of x, it is infeasible to come up with (Y^*, K^*) such that $(Y^*)^x = (Y^*)^{x'} = K^*$. Note that **this property does NOT hold for the Diffie–Hellman KEM as described above**, since the adversary can simply output $(Y^* = 1, K^* = 1)$. To fix this, we have to dictate that the identity group element *not* be a valid ciphertext, i.e., $\mathsf{Decap}(\star, 1)$ must output $\perp$. Then ROB is an information-theoretical property: such Y^* does not exist unless $x = x'$, which happens with negligible probability $1/q$.

Overall, we conclude that the Diffie–Hellman KEM has all desired properties under the CDH assumption, *except for ROB where we need to exclude* 1 *as a valid ciphertext*. Since ROB is needed in OEKE variants where both pw and pk are excluded from H_{tag} inputs (see Section 2.2), this precisely answers the question in Section 1: the "not hashing pw or pk" version of OEKE instantiated by Diffie–Hellman (**??**) is insecure, and adding *either* pw *or* pk fixes this issue.

Remark 2. The argument in the KBIND paragraph clearly requires the group order to be prime. The argument in the ROB paragraph also makes this requirement, although it can be straightforwardly extended to the general case where the group order has at least one large prime factor (a requirement we have to make anyway, since otherwise discrete logarithm is easy in the group due to the Pohlig–Hellman algorithm). None of the other properties requires the group order to be prime.

3 UC Description of the OEKE Protocol

We give a formal, UC-compatible description of the OEKE protocol in Figure 1. Note that UC PAKE considers the case that the two parties' passwords are different; therefore, we need to use different notations pw and pw' for the password of P and P', respectively.[5] We only describe the "hash everything" version of the protocol; in other variants one or more of pw, pk, (s, T) and c can be removed from the inputs to H_{tag}. We omit sid from RO inputs (e.g., we say "$H_1(\mathsf{pw}, r)$" rather than "$H_1(sid, \mathsf{pw}, r)$"), although we stress that sid must be included in order to achieve domain separation.

[5] [ABJ25, Fig.4] uses pw for both parties' passwords, which is not the "right" protocol description in the UC setting.

Let $\mathcal{G}$ be a PPT algorithm that outputs a (multiplicative) Abelian group $\mathbb{G}$ whose order is superpolynomial, and where uniformly sampling, multiplication and inverse can be done in polynomial time. (We do not require $\mathbb{G}$ to be cyclic, and do not assume $|\mathbb{G}|$ to be public.) We implicitly assume that $\mathbb{G} \leftarrow \mathcal{G}$ is run as part of the initialization process. Furthermore, we assume that all algorithms take $\mathbb{G}$ as an input, and do not explicitly write it.
Let $(\mathsf{Encap}, \mathsf{Decap})$ be a KEM scheme whose public-key space $\mathcal{PK} = \mathbb{G}$.

Party P, step 1

On input $(\mathsf{NewSession}, sid, P', \mathsf{pw}, \text{“initializer”})$, generate $(pk, sk) \leftarrow \mathsf{Gen}$, sample $r \leftarrow \{0,1\}^{3n}$, compute

$$R := H_1(\mathsf{pw}, r), \qquad T := pk \cdot R,$$
$$t := H_2(\mathsf{pw}, T), \qquad s := r \oplus t,$$

and send (sid, s, T) to P'. Store $\langle \mathsf{pw}, pk, sk, s, T\rangle$ associated with sid.

Party P'

On input $(\mathsf{NewSession}, sid, P, \mathsf{pw}', \text{“respondent”})$ and (sid, s, T) from P, compute

$$t' := H_2(\mathsf{pw}', T), \qquad r' := s \oplus t',$$
$$R' := H_1(\mathsf{pw}', r'), \qquad pk' := T/R',$$

$(c, K') \leftarrow \mathsf{Encap}(pk')$, $SK' := H_{\mathsf{key}}(K')$, and $\tau' := H_{\mathsf{tag}}(\mathsf{pw}', pk', s, T, c, K')$, send (sid, c, τ') to P, and output (sid, SK').

Party P, step 2

On (sid, c, τ') from P', retrieve $\langle \mathsf{pw}, pk, sk, s, T\rangle$ associated with sid. Then compute $K := \mathsf{Decap}(sk, c)$, $\tau := H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c, K)$, and

$$SK := \begin{cases} H_{\mathsf{key}}(K) & \text{if } \tau = \tau' \\ \perp & \text{if } \tau \neq \tau' \end{cases}$$

and output (sid, SK). // No need to store pw, pk or (s, T) in step 1 if not included in H_{tag} (but always need to store sk)

Figure 1: UC description of the OEKE protocol (the “hash everything” version)

Correctness. Suppose $\mathsf{pw} = \mathsf{pw}'$ and there is no adversary. Then P' computes

$$t' = H_2(\mathsf{pw}', T) = t, \qquad r' = s \oplus t' = r,$$
$$R' = H_1(\mathsf{pw}', r') = R, \qquad pk' = T/R' = pk,$$

so (c, K') is the result of $\mathsf{Encap}(pk')$. Then on the P side, P computes $K := \mathsf{Decap}(sk, c)$, which is equal to K' except with probability $\varepsilon_{\text{correctness}}$ (the correctness error of the KEM). If $K = K'$ then $\tau = \tau'$ so P's check passes, and

P will output $SK = SK'$. Therefore, the OEKE protocol has correctness error $\varepsilon_{\text{correctness}}$.

4 Security Analysis of the "Hash Everything" Version

The main theorem in this section is:

Theorem 1. *Suppose the UNI-PK, OW-1PCA and ANO-1PCA properties hold for the KEM scheme. Then the OEKE protocol as described in Figure 1 UC-realizes $\mathcal{F}_{\text{PAKE-1EA}}$, in the random oracle model for H_1, H_2, H_{key} and H_{tag}.*

The rest of this section is dedicated to the proof of Theorem 1. In Section 4.1 we describe the simulator, and in Section 4.2 we argue why the simulator generates a view indistinguishable from the real-world view.

As standard in UC proofs, w.l.o.g. we only construct a simulator $\mathcal{S}$ for the "dummy" adversary $\mathcal{A}$ that merely forwards all messages between the environment $\mathcal{Z}$ and protocol parties [Can01, Claim 11]. We omit *sid* in protocol messages and honest parties' outputs (e.g., we say "P outputs SK" rather than "P outputs (sid, SK)").

4.1 The Simulator

4.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Simulating honest P-to-P' message. When P's session begins, $\mathcal{S}$ must simulate its message (s, T) to P'. This can be easily done: just sample (s, T) at random. However, every time $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), $\mathcal{S}$ needs to generate a KEM key pair (pk^*, sk^*) for later use (see the "Extracting password guess from malicious P'" paragraph below) and make sure that (pk^*, sk^*) is consistent with the RO queries. This can be done as follows: $\mathcal{S}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*,$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. This also allows $\mathcal{S}$ to store the association between pw^* and (pk^*, sk^*). (Note that (pk^*, sk^*) needs to be freshly generated every time $\mathcal{A}$ queries $H_2(\star, T)$; otherwise R^* never changes and $\mathcal{A}$ can easily find collisions in H_1.)

Extracting password guess from malicious P. When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P', if it is different from the honest message (s, T), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding $\mathsf{TestPwd}$ command to $\mathcal{F}_{\text{PAKE-1EA}}$. There are two types of attack, which need to be addressed separately:

1. *"Normal" online attack:* $\mathcal{A}$ executes P's algorithm on a password guess pw^*. That is, $\mathcal{A}$ generates KEM public key pk^*, samples randomness r^*, computes

$$R^* := H_1(\mathsf{pw}^*, r^*), \qquad T^* := pk^* \cdot R^*,$$
$$t^* := H_2(\mathsf{pw}^*, T^*), \qquad s^* := r^* \oplus t^*,$$

and sends (s^*, T^*) to P'.
It takes a while to realize how to extract pw^* from (s^*, T^*). First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values; $\mathcal{S}$ must decide which one is the "actual" password guess. This can be done via the following. For every $H_2(\mathsf{pw}^*, T^*)$ query (let the result be t^*), $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\mathsf{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\mathsf{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the "actual" password guess pw^*.

2. *Related key attack:* This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P. $\mathcal{A}$ skips generating pk^* or sampling r^*; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$t^* := H_2(\mathsf{pw}^*, T), \qquad (t^*)' := H_2(\mathsf{pw}^*, T^*),$$
$$\delta := t^* \oplus (t')^*, \qquad s^* := s \oplus \delta,$$

and sends (s^*, T^*) to P'.
Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*), such that

$$s \oplus s^* = H_2(\mathsf{pw}, T) \oplus H_2(\mathsf{pw}, T^*),$$

which allows $\mathcal{A}$ to pick T^* and "reverse engineer" the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^*/H_1(\mathsf{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^*/T)$.
To extract pw^* from (s^*, T^*), $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which ones satisfy

$$H_2(\mathsf{pw}^*, T) \oplus H_2(\mathsf{pw}^*, T^*) = s \oplus s^*.$$

With overwhelming probability, at most one pw^* will satisfy this equation.

Remark: On the term "anchor query". The term "anchor query" was coined in [MRR20, Section 3.3], where it refers to the special $H_1(\mathsf{pw}^*, r^*)$ query made before the $H_2(\mathsf{pw}^*, T^*)$ query in case (1) above; case (2) is completely missing in [MRR20]. This error is fixed in [JRX25], which uses "anchor query" to refer to something else: the $H_2(\mathsf{pw}^*, T^*)$ query in either case (1) or (2) (see [JRX25, Section 4.3.1] and [JRX25, Figure 11, step 4]). This term is not used in [ABJ25].

Extracting password guess from malicious P'. When the adversary $\mathcal{A}$ sends a message (c^*, τ^*) to P, if $\mathcal{A}$ is not passive (i.e., it is not the case that $(s^*, T^*) = (s, T)$ *and* $(c^*, \tau^*) = (c, \tau)$), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding $\mathsf{TestPwd}$ command to $\mathcal{F}_{\text{PAKE-1EA}}$.

$\mathcal{S}$ should find the H_{tag} query resulting in τ^*, which contains $(\mathsf{pw}^*, pk^*, K^*)$. However, pw^* constitutes a password guess only if it is consistent with pk^* and K^*, i.e., $K^* = \mathsf{Decap}(sk^*, c^*)$ for sk^* corresponding to pk^*. $\mathcal{S}$ can check this condition because it has stored the correspondence between pw^* and (pk^*, sk^*) (see the "Simulating honest P-to-P' message" paragraph above), and can use sk^* to check if the equation holds. (This extraction strategy does not work anymore if *either* pw^* *or* pk^* is not included in H_{tag}. In later sections we will explain how to simulate other versions of the protocol.)

4.1.2 Formal Description of the Simulator

Below we use $\mathcal{F}$ as a shorthand for $\mathcal{F}_{\text{PAKE-1EA}}$. Please see Figures 2 and 3 for the simulator $\mathcal{S}$.

As notational conventions, we write "$\mathcal{S}$ sends message m from P to P'" as a shorthand for "$\mathcal{S}$ sends m to $\mathcal{Z}$ that pretends to be a real-world protocol message from P to P', intercepted by $\mathcal{A}$ which in turn forwards it to $\mathcal{Z}$". Similarly, we write "$\mathcal{S}$ receives message m from $\mathcal{A}$ to P" for "$\mathcal{Z}$ instructs $\mathcal{A}$ to send m to P, and in the ideal world it is $\mathcal{S}$ who receives m from $\mathcal{Z}$".

P-to-P' message

On $(\mathsf{NewSession}, sid, P, P', \text{"initiator"})$ from $\mathcal{F}$, sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$ and send (s, T) from P to P'.

P''s session key and P'-to-P message

On $(\mathsf{NewSession}, sid, P', P, \text{"respondent"})$ from $\mathcal{F}$ and (s^*, T^*) from $\mathcal{A}$ to P', simulate P''s output and the P'-to-P message as follows:

1. If $(s^*, T^*) = (s, T)$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$; generate $(pk', \star) \leftarrow \mathsf{Gen}$, compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$, sample $\tau' \leftarrow \{0,1\}^{2n}$, and send (c', τ') from P' to P.
2. Otherwise find $\mathcal{A}$'s $H_2((\mathsf{pw}')^*, T^*)$ query (let the result be t^*) that satisfies *either* of the following:
   - *Type (1):* $\mathcal{A}$ queried $H_1((\mathsf{pw}')^*, r^*)$ *before* the $H_2((\mathsf{pw}')^*, T^*)$ query, where $r^* = s^* \oplus t^*$.
   - *Type (2):* $\mathcal{A}$ also queried $H_2((\mathsf{pw}')^*, T)$, and
   $$s \oplus s^* = H_2((\mathsf{pw}')^*, T) \oplus t^*.$$
   (a) If there is more than one such $(\mathsf{pw}')^*$, abort.
   (b) If there is exactly one such $(\mathsf{pw}')^*$, send $(\mathsf{TestPwd}, sid, P', (\mathsf{pw}')^*)$ to $\mathcal{F}$; if there is no such $(\mathsf{pw}')^*$, send $(\mathsf{TestPwd}, sid, P', \perp)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$; sample $pk' \leftarrow \mathbb{G}$, compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$, sample $\tau' \leftarrow \{0,1\}^{2n}$, and send (c, τ') from P' to P.
      ii. If $\mathcal{F}$ replies with "correct guess", and $(\mathsf{pw}')^*$ is defined in type (1) above, then r^* is already defined. Compute
      $$R' := H_1((\mathsf{pw}')^*, r^*), \quad pk' := T^*/R'.$$
      If $\mathcal{F}$ replies with "correct guess", and $(\mathsf{pw}')^*$ is defined in type (2) above, then $\mathcal{A}$ has queried $H_2((\mathsf{pw}')^*, T)$, and a record $\langle (\mathsf{pw}')^*, (pk')^*, \star \rangle$ must have been stored at that time (see the "RO queries" paragraph below). Compute
      $$pk' := (pk')^* \cdot (T^*/T).$$
      Either way, compute $(c, K') \leftarrow \mathsf{Encap}(pk')$, $SK' := H_{\mathsf{key}}(K')$, $\tau' := H_{\mathsf{tag}}((\mathsf{pw}')^*, pk', s^*, T^*, c, K')$, and send (c, τ') from P' to P.

Figure 2: UC simulator $\mathcal{S}$ for OEKE (part 1). Red text denotes difference from the simulator in [ABJ25]

P's session key

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, s, T, c^*, K^*)$ query resulting in τ^*.
    (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
    (b) If there is exactly one such query, retrieve record $\langle \mathsf{pw}^*, \star, sk^* \rangle$. If there is no such record, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
    (c) Compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* \neq K$, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
    (d) Send $(\mathsf{TestPwd}, sid, P', \mathsf{pw}^*)$ to $\mathcal{F}$.
        i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$.
        ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P', SK)$ to $\mathcal{F}$.

RO queries

Answer all of $\mathcal{A}$'s RO queries via lazy sampling, except for the following:
When (s, T) is sampled (see the "P-to-P' message" paragraph above), if $\mathcal{A}$ has already queried $H_2(\star, T)$, abort.
On $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ query (let the result be t^*), generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$ and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$, and store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$.

Figure 3: UC simulator $\mathcal{S}$ for OEKE (part 2)

Our simulator is, by and large, the same as the one in [ABJ25, Fig.5 and 6], although we completely rewrite it to improve readability. For those who would like to go over both our simulator and the one in [ABJ25], we include a comparison in Section A.

4.2 Game-Hop Argument

We now prove that for any PPT environment $\mathcal{Z}$, the simulator $\mathcal{S}$ in Section 4.1.2 generates an ideal-world view indistinguishable from $\mathcal{Z}$'s real-world view. The sequence of games starts from the real world (Game 0) and ends in the ideal world (Game 17). We assume that $\mathcal{A}$ makes $q_1, q_2, q_{\mathsf{key}}, q_{\mathsf{tag}}$ queries to ROs $H_1, H_2, H_{\mathsf{key}}, H_{\mathsf{tag}}$, respectively. By Lemma 1, we may additionally assume that the $(q_1 + 1)(q_2 + 1)$-UNI-ANO-RO property holds for the KEM scheme. For two adjacent games i and $i+1$, let $\mathbf{Dist}_{\mathcal{Z}}^{i,i+1}$ be $\mathcal{Z}$'s distinguishing advantage between them.

A game-by-game comparison with the proof in [ABJ25, Section 5] is shown in *red*, with the parts that critically rely on certain items being included as inputs to H_{tag} *underlined*.

Game 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; the flow of events is:

- P sends (s, T) to P';
- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who sends (s^*, T^*) (which may or may not be equal to (s, T)) to P';
- P' outputs session key SK' and sends (c, τ') to P;
- It is again intercepted by $\mathcal{A}$, who sends (c^*, τ^*) to P;
- P computes τ. If $\tau = \tau^*$ then P outputs an n-bit string SK, otherwise P outputs $\perp$.

(Note that there are three τ values: τ' computed and sent by P', τ^* sent by $\mathcal{A}$, and τ computed by P used in P's check.) A graphic illustration is shown in Figure 4.

This game corresponds to game G_1 in [ABJ25].

P and P''s passwords match and $\mathcal{A}$ is passive, except maybe modifying the tag.

Game 1: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do the following:

- If $\tau^* = \tau'$ (i.e., $\mathcal{A}$ is passive), set $SK := SK'$.
- If $\tau^* \neq \tau'$, set $SK := \perp$.

By correctness of the protocol (see Section 3), in Game 0 $\tau = \tau'$ except with probability $\varepsilon_{\text{correctness}}$. This means that except with probability $\varepsilon_{\text{correctness}}$, P's check $\tau^* \stackrel{?}{=} \tau$ in Game 0 is equivalent to P's check $\tau^* \stackrel{?}{=} \tau'$ in Game 1. If the check passes then both games set $SK := H_{\mathsf{key}}(K)$, otherwise both games set $SK := \perp$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{0,1} = \varepsilon_{\text{correctness}}.$$

This game corresponds to games G_3–G_5 in [ABJ25]. We believe having a single game that handles both the $\tau^* = \tau'$ case and the $\tau^* \neq \tau'$ case is much cleaner.

$P(\mathsf{pw})$	$\mathcal{A}$	$P'(\mathsf{pw}')$
$(pk, sk) \leftarrow \mathsf{Gen}$		
$r \leftarrow \{0,1\}^{3n}$		
$R := H_1(\mathsf{pw}, r)$		
$T := pk \cdot R$		
$t := H_2(\mathsf{pw}, T)$		
$s := r \oplus t$		
	$\xrightarrow{s, T} \quad \xrightarrow{s^*, T^*}$	
		$t' := H_2(\mathsf{pw}', T^*)$
		$r' := s^* \oplus t'$
		$R' := H_1(\mathsf{pw}', r')$
		$pk' := T^*/R'$
		$(c, K') \leftarrow \mathsf{Encap}(pk')$
		$SK' := H_{\mathsf{key}}(K')$
		$\tau' := H_{\mathsf{tag}}(\mathsf{pw}', pk', s^*, T^*, c, K')$
	$\xleftarrow{c^*, \tau^*} \quad \xleftarrow{c, \tau'}$	
$K := \mathsf{Decap}(sk, c^*)$		
$\tau := H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$		
if $\tau \neq \tau^*$		
$\quad SK := \perp$		
otherwise		
$\quad SK := H_{\mathsf{key}}(K)$		
output SK		output SK'

Figure 4: Game 0 (the real world)

P and P''s passwords match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 2: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \bot$. If there is a unique such query, do the following:

- If $K^* = K$, set $SK := H_{\mathsf{key}}(K)$.
- If $K^* \neq K$, set $SK := \bot$.

Claim 1. *P' does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$.*

Proof. P' makes a single H_{tag} query, on $(\mathsf{pw}', pk', s^*, T^*, c, K')$. Since $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$.[6] □

Going back to the game hop, Game 1 and Game 2 are identical unless one of the following happens:

1. $\mathcal{A}$ never makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, \star)$ query resulting in τ^*, yet P's check $\tau^* \stackrel{?}{=} H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$ passes.
2. $\mathcal{A}$ makes more than one $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, \star)$ query resulting in τ^*.
3. $\mathcal{A}$ makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query (where $K^* \neq K$) resulting in τ^*, and P's $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$ query also results in τ^*.

In the first event, by Claim 1 P' does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$. $\mathcal{A}$ itself also does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$ (otherwise the result is τ^*). So $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$ is independent of $\mathcal{A}$'s view, and the probability that $\mathcal{A}$ sends $\tau^* = H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K)$ is $1/2^{2n}$. The second and third events are collision in H_{tag} from either $\mathcal{A}$'s or P's queries, whose (combined) probability is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} + 1)/2^{2n+1}$. Overall,

$$\mathbf{Dist}^{1,2}_{\mathcal{Z}} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}} + 1) + 2}{2^{2n+1}}.$$

This game corresponds to games G_6 and G_7 in [ABJ25], except that the H_{tag} collision cases (events (2) and (3) above) were ruled out in game G_2 in [ABJ25] (as aborting condition [A1]).

The proof of Claim 1 critically relies on pw, s, T, c being included in H_{tag} inputs.

Game 3: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as before.)

Let Query_1 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c, K')$. Clearly Game 2 and Game 3 are identical unless Query_1 happens. We

[6] The proof of this claim is trivial, but we explicitly present it as a claim since its counterparts in some variants are much harder to prove.

upper-bound $\Pr[\mathsf{Query}_1]$ via a reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property of the KEM scheme:

Reduction $\mathcal{R}_{\text{OW-1PCA1}}$: // $\boxed{\text{Boxed text}}$ indicates the single PC oracle query; same for reductions below

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_1.
On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk instead of $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW-1PCA1}}$'s input.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{key}}(K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}((K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_1 happens, the only difference between Game 2/3 and $\mathcal{R}_{\text{OW-1PCA1}}$'s simulation (until Query_1 happens) is that $\mathcal{R}_{\text{OW-1PCA1}}$ uses its input pk on the P side, and then uses its input $c \leftarrow \mathsf{Encap}(pk)$ on the P' side. Since we assume $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, in Game 2/3 P''s algorithm will recover $pk' = pk$, so there c is also computed as $\mathsf{Encap}(pk)$ and thus $\mathcal{R}_{\text{OW-1PCA1}}$ simulates Game 2/3 perfectly. Furthermore, if $\mathcal{R}_{\text{OW-1PCA1}}$'s guess is correct then it wins the OW-1PCA game. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{2,3} \leq \Pr[\mathsf{Query}_1] \leq (q_{\mathsf{key}} + q_{\mathsf{tag}}) \mathbf{Adv}_{\mathcal{R}_{\text{OW-1PCA1}}}^{\text{OW-1PCA}}.$$

This game corresponds to game G_8 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to OW-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than OW-1PCA. In this case, step 4 of the reduction above can query* $\mathsf{PC}(sk, c, (K')^*)$ $q_{\mathsf{key}} + q_{\mathsf{tag}}$ *times to check which of $\mathcal{A}$'s queries (if any) triggers* Query_1*, hence avoiding the* $q_{\mathsf{key}} + q_{\mathsf{tag}}$ *loss in its advantage.*

2. *The above also means that the plaintext-checking oracle in their OW-PCA game has to allow for* $\mathsf{PC}(sk, c, \star)$ *queries, whereas our reduction to OW-1PCA never makes a query in this form and thus the plaintext-checking oracle in our OW-1PCA game can disallow such queries.*

Game 4: Again in the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, generate P''s KEM public key $(pk', \star) \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$. (Note that K' is not used anywhere since Game 3 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.)

The difference between Game 3 and Game 4 is that in Game 3 both P and P' use the same pk, whereas in Game 4 P uses pk and P' uses an independent pk'. We construct a reduction $\mathcal{R}_{\text{ANO-1PCA}}$ to the ANO-1PCA property of the KEM scheme. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\text{ANO-1PCA}}$:

On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk instead of $(pk, \star) \leftarrow \mathsf{Gen}$. Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, output 1 and halt. Otherwise sample $SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO-PCA1}}$'s input.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$.

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{key}}(K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

$\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk on the P side and c on the P' side. If $\mathcal{R}_{\text{ANO-1PCA}}$'s challenge bit $b = 0$ then $c \leftarrow \mathsf{Encap}(pk)$, i.e., both P and P' use the same pk, so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 3; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., P' uses an independently generated pk', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 4. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{3,4} \leq \mathbf{Adv}_{\mathcal{R}_{\text{ANO-1PCA}}}^{\text{ANO-1PCA}}.$$

This game corresponds to game G_9 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to ANO-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than ANO-1PCA. Unlike in Game 3, here reducing to this stronger assumption does not provide any advantage.*
2. *The assumption [ABJ25] reduces to is even further stronger in that their KEM adversary's input is (pk, pk', c), whereas we do not give pk' to the KEM adversary. Note that the reduction does not need pk' at all (it only needs to send c which is an encapsulation of either pk or pk'), so the assumption in [ABJ25] is unnecessarily strong.*

P and P''s passwords do not match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 5: After P sends (s, T) aimed at P', when $\mathcal{A}$ makes a fresh query $H_2(\mathsf{pw}^*, T)$ for $\mathsf{pw}^* \neq \mathsf{pw}$ (let the result be t^*), generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$, and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$, and store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$.

Furthermore, in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, when P' queries $H_2(\mathsf{pw}', T)$, use the method described above to generate pk', r', R' and program H_1. (Note that the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ has already been addressed in Game 3 and Game 4, and P' does not need to make an H_2 query there.)[7]

In the analysis below, we combine $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ queries and P''s (possible) $H_2(\mathsf{pw}', T)$ query, and call all of them "$H_2(\mathsf{pw}^*, T)$". We first upper-bound the aborting probability. For every fresh $H_2(\mathsf{pw}^*, T)$ query, an independent $t^* \leftarrow \{0,1\}^{3n}$ value is sampled, which results in an independent $r^* \leftarrow \{0,1\}^{3n}$ (regardless of the distribution of s). As such, there are up to $q_2 + 1$ independent and uniformly random r^* values. The probability that one of $\mathcal{A}$'s q_1 $H_1(\mathsf{pw}^*, r^*)$

[7] A slightly confusing case is when $s^* \neq s \wedge T^* = T$, causing P' to query $t' := H_2(\mathsf{pw}', T)$; and $\mathcal{A}$ also queries $H_2(\mathsf{pw}', T)$. In this case P''s $r' = s^* \oplus t'$ and $\mathcal{A}$'s $r^* = s \oplus t'$ are different, causing P''s values R', pk' and $\mathcal{A}$'s values R^*, pk^* to be independent of each other. Therefore, the game should run P''s algorithm as usual when P' queries $H_2(\mathsf{pw}', T)$ (which generates R', pk'), and use the new method described in Game 5 when $\mathcal{A}$ makes the same query (which generates R^*, pk^*).

queries happens *before* the $H_2(\mathsf{pw}^*, T)$ query (when r^* is generated) is upper-bounded by $q_1(q_2+1)/2^{3n}$.[8]

Now suppose the aborting condition does not happen. The difference between Game 4 and Game 5 is that Game 4 samples $H_1(\mathsf{pw}^*, r^*)$ uniformly at random, whereas Game 5 defines it as T/pk^*. Note that

- For $H_2(\mathsf{pw}^*, T)$ queries where $\mathsf{pw}^* \neq \mathsf{pw}'$, sk^* (or even pk^*) is not used anywhere in the game;
- For the $H_2(\mathsf{pw}', T)$ query, we cannot say the above anymore, since pk' is used while computing c and K'. However, since $\mathsf{pw} \neq \mathsf{pw}'$, we know that $pk = T/H_1(\mathsf{pw}, r)$ is independent of $pk' = T/H_1(\mathsf{pw}', r')$, so sk' is still not used anywhere else in the game.

So, the distinguishing advantage between Game 4 and Game 5 can be reduced to the UNI-PK property of the KEM scheme. We only sketch the reduction $\mathcal{R}_{\text{UNI-PK}}$ here: $\mathcal{R}_{\text{UNI-PK}}$, on input pk^* (which is either an honestly generated public key or a uniformly random value), samples $i \leftarrow [q_2+1]$, and answers the (either $\mathcal{A}$'s or P''s) first $i-1$ H_2 queries as in Game 4 and the last q_2-i+1 H_2 queries as in Game 5; for the i-th H_2 query, $\mathcal{R}_{\text{UNI-PK}}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. If this H_2 query is on (pw', T), $\mathcal{R}_{\text{UNI-PK}}$ also uses its input pk^* to compute c and K'. (Note that the $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record is not actually used in Game 5, so $\mathcal{R}_{\text{UNI-PK}}$ does not need to store it.) $\mathcal{R}_{\text{UNI-PK}}$ simulates other parts of Game 4/5 honestly. Finally, $\mathcal{R}_{\text{UNI-PK}}$ copies $\mathcal{Z}$'s output bit.

A standard hybrid argument shows that (assuming the aborting condition does not happen) $\mathcal{Z}$'s distinguishing advantage between Game 4 and Game 5 is upper-bounded by $(q_2+1)\mathbf{Adv}^{\text{UNI-PK}}_{\mathcal{R}_{\text{UNI-PK}}}$.[9] Overall, we have

$$\mathbf{Dist}^{4,5}_{\mathcal{Z}} \leq (q_2+1)\mathbf{Adv}^{\text{UNI-PK}}_{\mathcal{R}_{\text{UNI-PK}}} + \frac{q_1(q_2+1)}{2^{3n}}.$$

This game corresponds to game G_{10} in [ABJ25]. However, there are two differences:

1. *[ABJ25] runs the procedure above to generate pk', r', R' and program H_1 as long as P' makes an $H_2(\mathsf{pw}', T)$ query — that is, including the case where $s^* \neq s \wedge T^* = T$. (If $T^* \neq T$ then of course P' does not make an $H_2(\mathsf{pw}', T)$ query and there is no change from the real world.) The reason*

---

[8] P also makes an H_1 query, but it is on (pw, r) where $\mathsf{pw} \neq \mathsf{pw}^*$ (so this H_1 query is impossible to trigger an abortion).

[9] Note that here we cannot make q_2+1 separate reductions to the UNI-PK property, and must reduce to UNI-PK "in one blow". Furthermore, the calculation of the reduction's advantage is more complicated than the normal case where the reduction wins as long as its guess is correct. This is similar to, say, why composing a PRG polynomial times yields another PRG; see, e.g., [BS23, Section 3.4.3] for details.

for making this change is to prepare for Game 6 and Game 7 below, which simplifies P''s algorithm when $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ (i.e., if we did not make the change in Game 5 first, the reductions in Game 6 and Game 7 would not go through); therefore, it is unnecessary (and potentially confusing) to make this change when $s^* \neq s \wedge T^* = T$.

2. *The aborting case was ruled out in game G_2 in [ABJ25] (as aborting condition [A2b1]).*

Game 6: In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as before.)

Let Query_2 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}', pk', s, T, c, K')$. Clearly Game 5 and Game 6 are identical unless Query_2 happens. We upper-bound $\Pr[\mathsf{Query}_2]$ via a reduction $\mathcal{R}_{\text{OW1}}$ to the OW property of the KEM scheme. This is somewhat similar to the reduction $\mathcal{R}_{\text{OW-1PCA}}$ to the OW-1PCA property in Game 3, and we highlight the differences in blue:

Reduction $\mathcal{R}_{\text{OW1}}$:

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_2, and $j \leftarrow [q_2 + 1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.
2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\text{OW1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\text{OW1}}$'s input pk' instead of generating pk^* freshly.
3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise
   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.
   (b) Sample $SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW1}}$'s input.)
4. On (c^*, τ^*) from $\mathcal{A}$,
   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)
   (b) Compute $K := \mathsf{Decap}(sk, c)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
   If $K = K^*$, output $SK := H_{\mathsf{key}}(K)$ to $\mathcal{Z}$.
   If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.

5. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}((K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}', pk', s, T, c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_2 happens and $\mathcal{R}_{\mathrm{OW1}}$'s guess of index j is correct, the only difference between Game 5/6 and $\mathcal{R}_{\mathrm{OW1}}$'s simulation (until Query_2 happens) is that $\mathcal{R}_{\mathrm{OW1}}$ uses its input pk' to program $H_1(\mathsf{pw}', r')$, and then uses its input $(c, \star) \leftarrow \mathsf{Encap}(pk')$ as the P'-to-P message. Since c is also computed as $\mathsf{Encap}(pk')$ in Game 5/6, $\mathcal{R}_{\mathrm{OW1}}$ simulates Game 5/6 perfectly. Furthermore, if $\mathcal{R}_{\mathrm{OW1}}$'s guess of index i is correct then it wins the OW game. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{5,6} \leq \Pr[\mathsf{Query}_2] \leq (q_2 + 1)(q_{\mathsf{key}} + q_{\mathsf{tag}})\mathbf{Adv}_{\mathcal{R}_{\mathrm{OW1}}}^{\mathrm{OW}}.$$

Remark 3. It is very subtle why the reduction needs to guess the index j and lose a factor of $q_2 + 1$ (and we applaud [ABJ25] for getting it right). The reason is as follows. Consider the following environment $\mathcal{Z}$:

1. Initiate P's session on password pw and receive (s, T).

2. Instruct $\mathcal{A}$ to make a large number of $H_2(\mathsf{pw}^*, T)$ queries (let the result be t^*), one of which is $H_2(\mathsf{pw}', T)$ for a special value pw'.

3. For each of the H_2 queries in step 2,

$$\text{compute } r^* := s \oplus t^*, \quad \text{query } H_1(\mathsf{pw}^*, r^*).$$

4. Initiate P''s session on password pw' and instruct $\mathcal{A}$ to forward (s, T) to P'.

To simulate Game 5/6 correctly, $\mathcal{R}_{\mathrm{OW1}}$ must use its input pk' to program $H_1(\mathsf{pw}', r') := T/pk'$ in step 3. The problem is that $\mathcal{R}_{\mathrm{OW1}}$ *does not know* pw' *until step 4, so it has no idea which* $H_1(\mathsf{pw}', r')$ *query in step 3 needs to be programmed using* pk'! The only way to get around is to make a guess over all $H_2(\star, T)$ queries, which incurs a loss up to $q_2 + 1$. (If we require $\mathcal{Z}$ to always initiate P and P''s sessions simultaneously, this loss can be avoided since in step 3 $\mathcal{R}_{\mathrm{OW1}}$ must have already learned pw'. However, the UC PAKE functionality does not place this restriction on $\mathcal{Z}$.)

This game corresponds to game G_{11} in [ABJ25], except that they reduce to OW-PCA and avoids the $q_{\mathsf{key}} + q_{\mathsf{tag}}$ loss (cf. the comment after Game 3).

Game 7: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, recall Game 6 does the following on the P' side: Generate $(pk', \star) \leftarrow \mathsf{Gen}$, query $t' := H_2(\mathsf{pw}', T)$, and compute

$$r' := s \oplus t', \quad R' := T/pk'.$$

((s, T) are the honest P-to-P' message.) If $H_1(\mathsf{pw}', r')$ has already been defined and it is not R', abort. Otherwise program $H_1(\mathsf{pw}', r') := R'$, $\underline{\text{compute } (c, \star) \leftarrow \mathsf{Encap}(pk')}$, and sample $SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$. Finally, output

SK' and send (c, τ') to P. (Note that K' is not used anywhere since Game 6 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.) In this game, we independently generate $(pk'', \star) \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk'')$ (replacing the underlined expression).

The difference between Game 6 and Game 7 is that in Game 6 c is computed from

$$pk' = \frac{T}{H_1(\mathsf{pw}', s \oplus H_2(\mathsf{pw}', T))}$$

which $\mathcal{A}$ can compute, whereas in Game 7 c is computed from an independently generated pk''. We construct a reduction $\mathcal{R}_{\mathrm{ANO}}$ to the ANO property of the KEM scheme. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\mathrm{OW1}}$ to the OW property in Game 6 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\mathrm{ANO}}$:

Sample $j \leftarrow [q_2 + 1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.

2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\mathrm{ANO1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\mathrm{ANO}}$'s input pk' instead of generating pk^* freshly.

3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, output 1 and halt. Otherwise

   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.

   (b) Sample $SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\mathrm{ANO}}$'s input.)

4. On (c^*, τ^*) from $\mathcal{A}$,

   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$.

   (b) Compute $K := \mathsf{Decap}(sk, c^*)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
   If $K = K^*$, output $SK := H_{\mathsf{key}}(K)$ to $\mathcal{Z}$.
   If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.

5. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

Assume $\mathcal{R}_{\text{ANO}}$'s guess of index j is correct. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., the computation of R' and c uses the same pk', so $\mathcal{R}_{\text{ANO}}$ simulates Game 6; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk'')$, i.e., the computation of c uses an independently generated pk'', so $\mathcal{R}_{\text{ANO}}$ simulates Game 7. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{6,7} \leq (q_2 + 1)\mathbf{Adv}_{\mathcal{R}_{\text{ANO}}}^{\text{ANO}}.$$

Game 8: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, we outright skip the computation of t', r', R' and the programming of H_1 (the brown text in Game 7). (Since pk' is not generated anymore, we can rename pk'' — the KEM public key from which c is computed — to pk'.)

Note that the aborting condition, which was originally introduced in Game 5, is removed (for the specific $H_2(\mathsf{pw}', T)$ query by P'). By an analysis similar to that in Game 5, the aborting probability in Game 7 is upper-bounded by $q_1/2^{3n}$. If the aborting condition does not happen, the brown text is independent of the rest of the game, so removing it does not affect the distribution of $\mathcal{Z}$'s output. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{7,8} \leq \frac{q_1}{2^{3n}}.$$

The two games above combined correspond to game G_{12} in [ABJ25]. [ABJ25] does not have the intermediate Game 7 in our proof, and outright removes the brown text from what's our Game 6. We believe it is clearer to include our Game 7, where H_2 is still queried and H_1 is still programmed: in this way it is easier to see why the reduction to ANO simulates the games correctly. Also, [ABJ25] reduces to ANO-PCA[10], although it also says that the plaintext-checking oracle is not needed.

Computation of the P-to-P' message.

Game 9: Note that up to now we have not changed P's algorithm before sending (s, T) (in particular, Game 5 dealt with H_2 queries by $\mathcal{A}$ and P', but not P). In this game we modify it as follows: On input ($\mathsf{NewSession}$, sid, P, P', pw, "initiator"), sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$. If $\mathcal{A}$ has queried $H_2(\star, T)$, abort. Otherwise query $t := H_2(\mathsf{pw}, T)$, generate $(pk, sk) \leftarrow \mathsf{Gen}$, and compute

$$r := s \oplus t, \quad R := T/pk.$$

If $H_1(\mathsf{pw}, r)$ has already been defined and it is not R, abort. Otherwise program $H_1(\mathsf{pw}, r) := R$ and store $\langle \mathsf{pw}, pk, sk \rangle$.

We first upper-bound the aborting probability. The probability that one of $\mathcal{A}$'s H_2 query inputs (before T is sampled) happens to contain T is upper-bounded by $q_2/|\mathbb{G}|$. By an analysis similar to that in Game 5 and Game 8, the probability of the second aborting event is upper-bounded by $q_1/2^{3n}$.

Now suppose neither of the aborting events happens. The following equations hold in both Game 8 and Game 9:

$$r = s \oplus H_2(\mathsf{pw}, T), \quad T = R \cdot pk.$$

[10] [ABJ25] actually mentions ANO-CPA, which appears to be a typo.

Here, pk is generated by Gen, and $H_2(\mathsf{pw}, T)$ is fixed by an H_2 query once T is fixed. The four remaining variables r, s, R, T are constrained by two equations and have $4 - 2 = 2$ degrees of freedom: if we sample $r \leftarrow \{0,1\}^{3n}, R \leftarrow \mathbb{G}$ then we get Game 8, and if we sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$ then we get Game 9. Thus, Game 8 and Game 9 are identical. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{8,9} \leq \frac{q_1}{2^{3n}} + \frac{q_2}{|\mathbb{G}|}.$$

The game above corresponds to game G_{13} in [ABJ25], with one difference: our game aborts if $\mathcal{A}$ has queried $H_2(\mathsf{pw}^*, T)$ for any pw^* when T is sampled, whereas the game in [ABJ25] aborts only if $\mathcal{A}$ has queried $H_2(\mathsf{pw}, T)$. Note that the simulator does not know pw, so it cannot tell if $\mathcal{A}$ has queried $H_2(\mathsf{pw}, T)$ and instead has to abort as long as $\mathcal{A}$ has queried $H_2(\star, T)$. (The simulator in [ABJ25] is correct, but its game hops do not match the ideal world regarding which H_2 queries will trigger an abortion.) Also, we believe our argument under this game is much more concise and cleaner (our presentation is, in spirit, similar to [JRX25, Lemma 4.5, Hybrid 2]).

Game 10: When P's session begins, halt after checking if $H_2(\star, T)$ has been defined (i.e., before the brown text in Game 9). Then when P receives (c^*, τ^*), if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do what's described in Game 1. Otherwise,

- If $\mathcal{A}$ has now queried $H_2(\mathsf{pw}, T)$, there must be a corresponding record $\langle \mathsf{pw}, pk, sk \rangle$ (see Game 5).
- If $\mathcal{A}$ still has not queried $H_2(\mathsf{pw}, T)$, run the brown text in Game 9.

Either way, pk, sk are defined and the game can proceed as usual.

The only difference between Game 9 and Game 10 lies in the fact that when we move the brown text around, the game might abort — as part of the brown text — at different times. Since the aborting event happens with probability at most $q_1/2^{3n}$, we have

$$\mathbf{Dist}_{\mathcal{Z}}^{9,10} \leq \frac{q_1}{2^{3n}}.$$

The game above corresponds to game G_{14} in [ABJ25]. Note that we have a $q_1/2^{3n}$ loss in both Game 9 and Game 10; if we remove Game 9 and directly jump to Game 10 from Game 8, this loss does not need to be repeated (since Game 8 and Game 10 are identical unless one of the two aborting events happens). We (as well as [ABJ25]) choose to include Game 9 for clarity.

Intermission. Let's pause a bit and review the changes we have made so far:

1. *P-to-P' message:* Game 10 outright samples (s, T) at random. [changed in Game 9]
2. *P''s session key and P'-to-P message:* On (s^*, T^*) from $\mathcal{A}$ to P', if $(s^*, T^*) = (s, T)$, Game 10 outright samples SK', τ' at random. Regarding

c, Game 10 generates $(pk', \star) \leftarrow \mathsf{Gen}$ and computes $(c, \star) \leftarrow \mathsf{Encap}(pk')$. (If $(s^*, T^*) \neq (s, T)$ then there is no change from the real world.) [changed in Games 3, 4, 6, 7, 8]

3. *P's session key:* On (c^*, τ^*) from $\mathcal{A}$ to P, we have made the following changes:

   (a) If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, Game 10 outright sets $SK := SK'$ if $\tau^* = \tau'$ and $SK := \perp$ otherwise. [changed in Game 1]

   (b) Otherwise if $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query resulting in τ^*, or more than one such query, Game 10 outright sets $SK := \perp$. [changed in Game 2]

   (c) Otherwise if sk has not been defined yet, Game 10 queries $H_2(\mathsf{pw}, T)$ on behalf of P, which generates sk (see below). The remaining part is unchanged from the real world.

4. *H_2 queries:* When either $\mathcal{A}$ or P makes a fresh query $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), Game 10 runs the following procedure: Generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$, and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

   If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$. [changed in Games 5, 10]

See Figure 5 for a summary of Game 10.

Password extraction from P'-to-P message.

Game 11: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, if there is no $\langle \mathsf{pw}, \star, \star \rangle$ record, set $SK := \perp$ (i.e., replace the brown text in Figure 5 with "$SK := \perp$").

Game 10 and Game 11 differ only if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query, but makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query resulting in τ^* (for c^*, K^*, τ^* of $\mathcal{A}$'s choice). This in particular requires $\mathcal{A}$ to know pk, which is freshly generated by Gen. Therefore, the probability that (even a computationally unbounded) $\mathcal{A}$ can predict pk is upper-bounded by $p_{\text{guess}}(pk)$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{10,11} \leq p_{\text{guess}}(pk).$$

The game above corresponds to game G_{15} in [ABJ25]. [ABJ25] does not include the condition $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, i.e., it appears that this game rewrites what our Game 2 does, which is incorrect.

[ABJ25] uses the notation $H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen})$ instead of $p_{guess}(pk)$, without explaining what it means. By default it is the min-entropy of pk, namely $-\log p_{guess}(pk)$ (which is a value greater than 1). But this is clearly not what [ABJ25] means.

$P(\mathsf{pw})$		$\mathcal{A}$		$P'(\mathsf{pw}')$
$s \leftarrow \{0,1\}^{3n}; T \leftarrow \mathbb{G}$				
	$\xrightarrow{s, T}$		$\xrightarrow{s^*, T^*}$	
				if $(s^*, T^*) = (s, T)$
				$\quad SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$
				$\quad (pk', \star) \leftarrow \mathsf{Gen}$
				$\quad (c, \star) \leftarrow \mathsf{Encap}(pk')$
				otherwise
				$\quad t' := H_2(\mathsf{pw}', T^*)$
				$\quad r' := s^* \oplus t'$
				$\quad R' := H_1(\mathsf{pw}', r')$
				$\quad pk' := T^*/R'$
				$\quad (c, K') \leftarrow \mathsf{Encap}(pk')$
				$\quad SK' := H_{\mathsf{key}}(K')$
				$\quad \tau' := H_{\mathsf{tag}}(\mathsf{pw}', pk', s^*, T^*, c, K')$
	$\xleftarrow{c^*, \tau^*}$		$\xleftarrow{c, \tau'}$	
if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$				
$\quad$ if $\tau^* = \tau$				
$\quad\quad SK := SK'$				
$\quad$ otherwise				
$\quad\quad SK := \perp$				
otherwise				
$\quad$ retrieve $\langle \mathsf{pw}, pk, sk \rangle$ record				
$\quad$ if none				
$\quad\quad t := H_2(\mathsf{pw}, T)$				
$\quad\quad (pk, sk) \leftarrow \mathsf{Gen}$				
$\quad\quad r := s \oplus t$				
$\quad\quad R := T/pk$				
$\quad\quad H_1(\mathsf{pw}, r) := R$				
$\quad\quad$ store $\langle \mathsf{pw}, pk, sk \rangle$				
$\quad$ find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*) = \tau^*$ query				
$\quad$ if none or more than one				
$\quad\quad SK := \perp$				
$\quad K := \mathsf{Decap}(sk, c^*)$				
$\quad$ if $K^* = K$				
$\quad\quad SK := H_{\mathsf{key}}(K)$				
$\quad$ otherwise				
$\quad\quad SK := \perp$				
output SK				output SK'
on $\mathcal{A}$'s fresh $H_2(\mathsf{pw}^*, T) =: t^*$ query:				
if there is no $\langle \mathsf{pw}^*, \star, \star \rangle$ record				
$\quad (pk^*, sk^*) \leftarrow \mathsf{Gen}$				
$\quad r^* := s \oplus t^*$				
$\quad R^* := T/pk^*$				
$\quad H_1(\mathsf{pw}^*, r^*) := R^*$				
$\quad$ store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$				

Figure 5: Game 10 (with various aborting conditions omitted)

The argument in this game critically relies on pk being included in H_{tag} inputs.

Game 12: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$, do what's in the $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$ case (i.e., replace line 21 of Figure 5 with line 23–38).

In Game 11 SK is outright set to $\perp$, whereas here it is treated the same as the "normal" case. Let Query_3 be the event that $\mathcal{A}$ queries $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c, K)$, where pk is the KEM public key generated on $\mathcal{A}$'s $H_2(\mathsf{pw}, T)$ query; Game 11 and Game 12 are identical unless Query_3 happens. To trigger Query_3, $\mathcal{A}$ must know K, which suggests a reduction $\mathcal{R}_{\mathrm{OW2}}$ to the OW property of the KEM scheme. This reduction is simple, so we only provide a sketch: $\mathcal{R}_{\mathrm{OW2}}$, on input (pk, c), invokes $\mathcal{Z}$ and simulates (s, T) and the P' side honestly. (Note that P' uses a freshly generated pk' independent of the P side.) When $\mathcal{A}$ queries $H_2(\mathsf{pw}, T)$, $\mathcal{R}_{\mathrm{OW2}}$ uses its input pk in lieu of $(pk, \star) \leftarrow \mathsf{Gen}$. Along the way, as long as $\mathcal{R}_{\mathrm{OW2}}$ detects that $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$ does not happen, it aborts. Finally, $\mathcal{R}_{\mathrm{OW2}}$ finds $\mathcal{A}$'s (unique) $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c, K^*)$ query resulting in τ^*, and outputs K^*. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{11,12} \leq \Pr[\mathsf{Query}_3] \leq \mathbf{Adv}_{\mathcal{R}_{\mathrm{OW2}}}^{\mathrm{OW}}.$$

The game above is missing in [ABJ25]. Note that in the ideal world, the simulator cannot tell if $\mathsf{pw} = \mathsf{pw}'$, so when it sees $(s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$, it has to always try to extract a password guess; hence, the game above has to be here. (In the $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$ case, we first outright set $SK := \perp$ in Game 1 and then "walk back" in Game 12, since the change in Game 1 is necessary for the reductions in Games 3 and 4 to go through.)

Game 13: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau))$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, s, T, c^*, K^*)$ query resulting in τ^*. (The difference is that Game 12 only goes over all of $\mathcal{A}$'s queries in the form of $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, \star)$, whereas this game goes over all of $\mathcal{A}$'s queries in the form of $H_{\mathsf{tag}}(\star, \star, s, T, c^*, \star)$.) If there is none or more than one, set $SK := \perp$. Otherwise there is a unique such $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, s, T, c^*, K^*)$ query, and

- If $\mathsf{pw}^* \neq \mathsf{pw}$, set $SK := \perp$.
- If $\mathsf{pw}^* = \mathsf{pw}$, there is no change from Game 12.

If $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\star, \star, s, T, c^*, \star)$ query resulting in τ^*, then in particular $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, \star)$ query resulting in τ^*, so both Game 12 and Game 13 set $SK := \perp$. The probability that $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\star, \star, s, T, c^*, \star)$ queries both resulting in τ^* is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)/2^{2n+1}$. Now suppose $\mathcal{A}$ makes a unique $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, s, T, c^*, K^*)$ query resulting in τ^*:

- If $\mathsf{pw}^* \neq \mathsf{pw}$, then $\mathcal{A}$ does not make a $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, \star)$ query resulting in τ^* (because there is only one $H_{\mathsf{tag}}(\star, \star, s, T, c^*, \star)$ query resulting in τ^*, and it is on pw^*), so both Game 12 and Game 13 set $SK := \perp$.

- If $\mathsf{pw}^* = \mathsf{pw}$, then there is no difference between Game 12 and Game 13.

We conclude that

$$\mathbf{Dist}_{\mathcal{Z}}^{12,13} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)}{2^{2n+1}}.$$

The game above is also missing in [ABJ25]. The H_{tag} collision case was ruled out in game G_2 in [ABJ25] (as aborting condition [A1])[11]*; however, conceptually speaking there still needs to be such a bridge game, to make sure that the game hops match the ideal world at the end.*

The definition of this game, together with its argument, critically rely on pw being included in H_{tag} inputs.

Game 14: In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, set $SK := \perp$.

In this case $\tau^* = \tau'$ is uniformly random and independent of the rest of the game, so the probability that $\mathcal{A}$ makes an H_{tag} query resulting in τ^* is at most $q_{\mathsf{tag}}/2^{2n}$. If $\mathcal{A}$ does not make such a query, then both Game 13 and Game 14 set $SK := \perp$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{13,14} \leq \frac{q_{\mathsf{tag}}}{2^{2n}}.$$

The game above is also missing in [ABJ25]. Again in the ideal world, the simulator cannot tell if $\mathsf{pw} = \mathsf{pw}'$, so when it sees $\mathcal{A}$ is passive, it has to let the PAKE functionality check if $\mathsf{pw} = \mathsf{pw}'$ and set SK to be equal to SK' or $\perp$ accordingly; hence, the game above has to be here.

Password extraction from P-to-P' message.

Game 15: In the case where $(s^*, T^*) \neq (s, T)$, find $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T^*)$ query[12] (let the result be t^*) that satisfies *either* of the following:

1. $\mathcal{A}$ queried $H_1(\mathsf{pw}^*, r^*)$ *before* the $H_2(\mathsf{pw}^*, T^*)$ query, where $r^* = s^* \oplus t^*$.
2. $\mathcal{A}$ also queried $H_2(\mathsf{pw}^*, T)$, and

$$s \oplus s^* = H_2(\mathsf{pw}^*, T) \oplus t^*.$$ [13]

(This $H_2(\mathsf{pw}^*, T^*)$ query is what's called the "anchor query" in [JRX25].) If there is more than one such query, abort.

The aborting condition is met only in the following three cases:

[11] In our game hops, we ruled out H_{tag} collisions for different K^* values in Game 2, and then additionally rule out H_{tag} collisions for different $(\mathsf{pw}^*, pk^*, K^*)$ tuples in Game 13. We could have done both in Game 2 and avoided the additional loss here; however, we believe the current approach is clearer.

[12] The notations are slightly messed up, since the pw^* here has nothing to do with what's called pw^* in Game 13. Essentially, the pw^* here is $\mathcal{A}$'s guess of P''s password (which is called $(\mathsf{pw}')^*$ in the simulator), whereas the pw^* in Game 13 is $\mathcal{A}$'s guess of P's password. The good news is that the pw^* in Game 13 is never reused once Game 13 is done, so there is no potential confusion.

[13] Note that in this case $T^* \neq T$ since otherwise $T^* = T \Rightarrow H_2(\mathsf{pw}^*, T) = t^* \Rightarrow s^* = s \oplus H_2(\mathsf{pw}^*, T) \oplus t^* = s$, so $(s^*, T^*) = (s, T)$ which violates the condition of this game.

- There is a collision in type (1) above, i.e., there are $r^*, t^*, (r^*)', (t^*)'$ such that
$$r^* \oplus t^* = (r^*)' \oplus (t^*)'$$
(so $\mathcal{A}$ can set s^* to be this value). Note that all four values in the equation are determined by two (different) H_1 queries and one H_2 query, and for each set of these queries, there is a $1/2^{3n}$ chance that the equation will hold. Therefore, the probability of this case is upper-bounded by $q_1(q_1 - 1)q_2/2^{3n}$.

- There is a collision in type (2) above, i.e., there are $\mathsf{pw}^*, t^*, (\mathsf{pw}^*)', (t^*)'$ such that
$$H_2(\mathsf{pw}^*, T) \oplus t^* = H_2((\mathsf{pw}^*)', T) \oplus (t^*)'$$
(so $\mathcal{A}$ can set s^* to be $s \oplus H_2(\mathsf{pw}^*, T) \oplus t^*$). Note that all four values in the equation are determined by three (different) H_2 queries $H_2(\mathsf{pw}^*, T)$, $H_2((\mathsf{pw}^*)', T)$ and $H_2(\mathsf{pw}^*, T^*)$ (they are different since $T^* \neq T$); by an argument similar to above, the probability of this case is upper-bounded by $q_2(q_2 - 1)(q_2 - 2)/2^{3n}$.

- There is a collision between type (1) and type (2), i.e., there are $r^*, t^*, (\mathsf{pw}^*)'$, $(t^*)'$ such that
$$r^* \oplus t^* = s \oplus H_2((\mathsf{pw}^*)', T) \oplus (t^*)'$$
(so $\mathcal{A}$ can set s^* to be this value). Note that s is already fixed, and all four other values in the equation are determined by two (different) H_2 queries and one H_1 query; by an argument similar to above, the probability of this case is upper-bounded by $q_1 q_2(q_2 - 1)/2^{3n}$.

Summing up, we have
$$\mathbf{Dist}_{\mathcal{Z}}^{14,15} \leq \frac{q_2[q_1(q_1 + q_2 - 2) + (q_2 - 1)(q_2 - 2)]}{2^{3n}}.$$

This game corresponds to part of game G_2 in [ABJ25] (as aborting conditions [A2a] and [A2b2]). The game in [ABJ25] aborts more often than necessary, though: every time $\mathcal{A}$ makes an $H_2(\star, \star)$ query, the game performs the checks and aborts accordingly, while it is only necessary to go over all of $\mathcal{A}$'s $H_2(\star, T^*)$ queries for the specific T^* as part of $\mathcal{A}$'s message to P'.

Performing the checks on $\mathcal{A}$'s H_2 queries, instead of when $\mathcal{A}$ sends the (s^*, T^*) message, also adds more bookkeeping and complicates the proof. In [ABJ25], after performing the checks, the game (if it does not abort) computes the corresponding KEM public key and stores it together with (pw^*, T^*) (the record is called ForwS); later when $\mathcal{A}$ sends (s^*, T^*), the game first fetches the record to recover pw^*, and if $\mathsf{pw}^* = \mathsf{pw}'$, the game then recovers the KEM public key and uses it to complete the remaining part of P''s algorithm. This creates a conceptual discrepancy with the ideal world, and a game hop (game G_{16} in [ABJ25]) is needed. By performing the checks when $\mathcal{A}$ sends (s^*, T^*), we avoid all these and skip their game G_{16}.

Correct password guess for P'.

Game 16: Again in the case where $(s^*, T^*) \neq (s, T)$, if pw^* is defined in type (2) in Game 15 and $\mathsf{pw}^* = \mathsf{pw}'$, retrieve record $\langle \mathsf{pw}^*, pk^*, \star\rangle$ (there must be one since such a record is created when $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$, as per Game 5) and set $pk' := pk^* \cdot (T^*/T)$.[14]

In Game 15, we have (in P''s algorithm)

$$pk' = \frac{T^*}{R'} = \frac{T^*}{H_1(\mathsf{pw}', r')} = \frac{T^*}{H_1(\mathsf{pw}', s^* \oplus t')} = \frac{T^*}{H_1(\mathsf{pw}', s^* \oplus H_2(\mathsf{pw}', T^*))},$$

but also (in the creation of the record $\langle \mathsf{pw}^*, pk^*, \star\rangle$)

$$pk^* = \frac{T}{R^*} = \frac{T}{H_1(\mathsf{pw}^*, r^*)} = \frac{T}{H_1(\mathsf{pw}^*, s \oplus t^*)} = \frac{T}{H_1(\mathsf{pw}^*, s \oplus H_2(\mathsf{pw}^*, T))}.$$

The condition of this game specifies that $\mathsf{pw}^* = \mathsf{pw}'$, and type (2) in Game 15 specifies that $s \oplus H_2(\mathsf{pw}^*, T) = s \oplus H_2(\mathsf{pw}^*, T^*)$. This means that the denominators in the expressions of pk' and pk^* are equal, and thus $pk' = pk^* \cdot (T^*/T)$ — exactly as in Game 15. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{15,16} = 0.$$

This game corresponds to game G_{17} in [ABJ25].

Incorrect password guess for P'.

Game 17: Again in the case where $(s^*, T^*) \neq (s, T)$, if $\mathsf{pw}^* \neq \mathsf{pw}'$ or no pw^* is defined as in Game 15, sample $pk' \leftarrow \mathbb{G}$, compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$, and sample $SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$.

We construct a reduction $\mathcal{R}_{\text{UNI-ANO-RO}}$ to the $(q_1+1)(q_2+1)$-UNI-ANO-RO property of the KEM scheme:

Reduction $\mathcal{R}_{\text{UNI-ANO-RO}}$:

On input $(pk_{1,1}, \ldots, pk_{q_1+1,q_2+1})$,

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$. Send (s, T) to $\mathcal{A}$.

2. Whenever there is an $H_2(\star, T)$ query by $\mathcal{A}$, do what's described in Game 5 (i.e., line 42–46 in Figure 5). (This in particular yields a record in the form of $\langle \mathsf{pw}^*, pk^*, sk^*\rangle$.) Furthermore, for the j-th query to H_2 (by $\mathcal{A}$ or $\mathcal{R}_{\text{UNI-ANO-RO}}$ itself on behalf of P' — see step 3 below), say it is $H_2(\mathsf{pw}^*, T^*) =: t^*$, and the i-th H_1 query in the form of $H_1(\mathsf{pw}^*, r^*)$, compute
$$s^* := r^* \oplus t^*, \quad R^* := T^*/pk_{i,j}.$$

[14] What's called pk^* here is $(pk')^*$ in the simulator, as there is another pk^* on the P side as defined in Game 13 (cf. Footnote 12).

(a) If $H_1(\mathsf{pw}^*, r^*)$ has been defined when the $H_2(\mathsf{pw}^*, T^*)$ query happens (either because it was queried *before* the $H_2(\mathsf{pw}^*, T^*) =: t^*$ query or because it has been programmed by $\mathcal{R}_{\text{UNI-ANO-RO}}$ while answering $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ query), store $\langle \mathsf{PasswordGuess}, \mathsf{pw}^*, s^*, T^* \rangle$. // Anchor query, i.e., pw^* is the password guess corresponding to (s^*, T^*)
At any time, if there is more than one $\langle \mathsf{PasswordGuess}, \star, s^*, T^* \rangle$ record with the same s^*, T^*, abort.

(b) Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$.

3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $(s^*, T^*) = (s, T)$, output 1. If there is a record $\langle \mathsf{PasswordGuess}, \mathsf{pw}', s^*, T^* \rangle$, also output 1. Otherwise // Incorrect or no password guess (the case that matters in this game hop)

   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T^*)$ or $H_1(\mathsf{pw}', r')$ (where $r' = s^* \oplus H_2(\mathsf{pw}', T^*)$), make these queries on behalf of P'.

   (b) Say $H_2(\mathsf{pw}', T^*)$ is the j^*-th query to H_2 and $H_1(\mathsf{pw}', r')$ is the i^*-th query to H_1 in the form of $H_1(\mathsf{pw}', \star)$. Output (i^*, j^*) to the challenger and receive (c, SK', τ').

   (c) Output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$.

4. On (c^*, τ^*) from $\mathcal{A}$,

   (a) Retrieve record $\langle \mathsf{pw}, pk, sk \rangle$ stored in step 2. If there is no such record, output $SK := \perp$ to $\mathcal{Z}$.

   (b) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$.

   (c) Compute $K := \mathsf{Decap}(sk, c^*)$. // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
   If $K = K^*$, output $SK := H_{\mathsf{key}}(K)$ to $\mathcal{Z}$.
   If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.

5. When $\mathcal{Z}$ outputs a bit b^*, copy $\mathcal{Z}$'s output.

We first argue that the pw^* as in record $\langle \mathsf{PasswordGuess}, \mathsf{pw}^*, s^*, T^* \rangle$ is exactly the same as the pw^* extracted from (s^*, T^*) in Game 15:

- If $\mathcal{A}$ queried $H_1(\mathsf{pw}^*, r^*)$ before the $H_2(\mathsf{pw}^*, T^*)$ query, this is exactly type (1) in Game 15.

- If $\mathcal{A}$ queried $H_2(\mathsf{pw}^*, T)$, which triggered the programming of $H_1(\mathsf{pw}^*, r^*)$ before the $H_2(\mathsf{pw}^*, T^*)$ query, the code in Game 5 (i.e., line 42–46 in Figure 5) specifies that the H_1 instance that is programmed on is $(\mathsf{pw}^*, s \oplus H_2(\mathsf{pw}^*, T))$. Thus,

$$s \oplus H_2(\mathsf{pw}^*, T) = r^* = s^* \oplus H_2(\mathsf{pw}^*, T^*),$$

  which is exactly type (2) in Game 15.

Therefore, $\mathcal{R}_{\text{UNI-ANO-RO}}$ accurately decides whether the condition "$\mathsf{pw}^* \neq \mathsf{pw}'$ or no pw^* is defined" is satisfied or not.

Next, we argue that $\mathcal{R}_{\text{UNI-ANO-RO}}$ simulates Game 16/17 correctly:

- Suppose $\mathcal{R}_{\text{UNI-ANO-RO}}$'s challenge bit $b = 0$, i.e., $(c, K') \leftarrow \mathsf{Encap}(pk_{i^*,j^*})$ and $SK' = H_{\mathsf{key}}(K'), \tau' = H_{\mathsf{tag}}(\ldots, K')$. For the "special" $H_2(\mathsf{pw}', T^*)$ query and $H_1(\mathsf{pw}', r')$ query, in both Game 16 and $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation,
$$s^* = r' \oplus H_2(\mathsf{pw}', T^*), \quad t^* = pk' \cdot H_1(\mathsf{pw}', r');$$
if we sample $H_1(\mathsf{pw}', r') \leftarrow \mathbb{G}$ then we get Game 16, whereas if we sample $pk' \leftarrow \mathbb{G}$ (as pk_{i^*,j^*}) then we get $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation. Thus, the distribution of pk' is identical in Game 16 and in $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation. Later, pk' is used to compute c, K' in both Game 16 and $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation.
Regarding other $H_2(\mathsf{pw}^*, T^*)$ and $H_1(\mathsf{pw}^*, r^*)$ queries, the $pk_{i,j}$ that $\mathcal{R}_{\text{UNI-ANO-RO}}$ uses while programming $H_1(\mathsf{pw}^*, r^*)$ is uniformly random and independent of the rest of the game, so $H_1(\mathsf{pw}^*, r^*)$ is programmed to a uniformly random value — again the same as Game 16.

- Suppose $\mathcal{R}_{\text{UNI-ANO-RO}}$'s challenge bit $b = 1$, i.e., $(c, K') \leftarrow \mathsf{Encap}(pk')$ and $SK' = H_{\mathsf{key}}(K'), \tau' = H_{\mathsf{tag}}(\ldots, K')$ for a freshly sampled pk'. *All* $pk_{i,j}$ values that $\mathcal{R}_{\text{UNI-ANO-RO}}$ uses while programming H_1 are uniformly random and independent of the rest of the game, so H_1 is always programmed to a uniformly random value — the same as Game 17. Later, a freshly sampled pk' is used to compute c, K' in both Game 17 and $\mathcal{R}_{\text{UNI-ANO-RO}}$'s simulation.

We have
$$\mathbf{Dist}_{\mathcal{Z}}^{16,17} \leq \mathbf{Adv}_{\mathcal{R}_{\text{UNI-ANO-RO}}}^{(q_1+1)(q_2+1)\text{-UNI-ANO-RO}}.$$

This game corresponds to games G_{18} and G_{19} in [ABJ25]. [ABJ25] has two games: G_{18} that changes SK' and τ' (via a reduction to the OW-PCA property of the KEM scheme), and G_{19} that changes c (via a reduction to the ANO-PCA property of the KEM scheme). While we believe their argument is fundamentally correct, it could be significantly improved:

1. *[ABJ25] reduces to OW-PCA and ANO-PCA[15], which is unnecessary: OW and ANO suffice.*

2. *Their simulation strategy is slightly different from ours: they use pk' generated by Gen, rather than sampled uniformly at random. Note that in this "incorrect password guess" case, in the real world $pk' = T^*/H_1(\mathsf{pw}', r')$ is already uniformly random, so this is fundamentally a KEM property about*

[15] [ABJ25] actually mentions ANO-CPA, which appears to be a typo.

encapsulation of uniformly random (rather than honestly generated) public keys and we believe it is clearer to present it in this way.[16]

3. *Their honest generation of pk' also means that their reductions cannot simulate the games perfectly, since they program $H_1(\mathsf{pw}', r')$ using pk' which is not uniformly random — while it "should" be uniformly random. As such, they have do a reduction to UNI-PK within the OW-PCA/ANO-PCA reduction.*

4. *What makes things even worse is that none of their (or our) reductions knows which of the $H_1(\mathsf{pw}', r')$ queries should be programmed using pk', since $\mathcal{A}$ can make many decryption efforts, resulting in many H_1 values to be programmed,* before *sending (s^*, T^*) to P' (cf. discussion after Definition 6). This means that their UNI-PK reduction cannot be moved "out of" the OW-PCA/ANO-PCA reduction by defining a separate game that uses honestly generated pk' to program $H_1(\mathsf{pw}', r')$, as that would require* the game *to make a guess! This "reduction within a reduction" would better be filtered out as a separate helper security property (cf. our Lemmas 2 and 3), which is not mentioned in [ABJ25].*

5. *Finally, the reductions in [ABJ25] appear to never check for the "related key attack", i.e., from their proof it is unclear what will go wrong if we remove what's our Game 16 (or their game G_{17}).*

We believe it is much cleaner to define a tailored KEM property that allows a single *reduction to go through, which is UNI-ANO-RO. In this way, we can separate out the multiple reductions to basic KEM properties, as well as the hybrid argument, on the level of KEM; this makes our OEKE security argument easier to understand.*[17]

Summary. We summarize the changes since Game 11 (i.e., after Figure 5):

1. *P''s session key and P'-to-P message:* On (s^*, T^*) from $\mathcal{A}$ to P', if $(s^*, T^*) \neq (s, T)$, Game 17 extracts $(\mathsf{pw}')^*$ from (s^*, T^*) using the method in Game 15. Then

   (a) If $(\mathsf{pw}')^* \neq \mathsf{pw}'$, then Game 17 outright samples SK', τ' at random. Regarding c, Game 17 samples $pk' \leftarrow \mathbb{G}$ and computes $(c, \star) \leftarrow \mathsf{Encap}(pk')$. [changed in Game 17]

[16] By contrast, in the $(s^*, T^*) = (s, T)$ case pk' has to be generated honestly, which is our Games 3, 4, 6 and 7: If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ then sk corresponding to $pk = pk'$ is used on the P side (in decapsulation), so pk' cannot be changed to uniformly random; and the $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ case cannot be separated from the $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ case (since the simulator never knows if $\mathsf{pw} = \mathsf{pw}'$ or not), so pk' cannot be changed to uniformly random in this case either.

[17] In principle, we could also combine Game 3 and Game 4, as well as Game 6 and Game 7, via a single reduction to some "integrated" KEM properties. We choose not to do it as having many new, tailored KEM properties could be confusing.

(b) If $(\mathsf{pw}')^* = \mathsf{pw}'$, Game 15 specifies two types of extraction of $(\mathsf{pw}')^*$. There is no change from the real world for type (1). For type (2), Game 17 computes $pk' := (pk')^* \cdot (T^*/T)$ (and uses pk' to compute c', SK', τ'). [changed in Game 16]

2. *P's session key:* On (c^*, τ^*) from $\mathcal{A}$ to P, we have made the following changes:

   (a) If $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, Game 17 outright sets $SK := \perp$. [changed in Game 14]

   (b) If $\neg((s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau'))$ (i.e., $\mathcal{A}$ is not passive), Game 17 retrieves two records:

   - $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, s, T, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, or if there is a unique such query with $\mathsf{pw}^* \neq \mathsf{pw}$, Game 17 sets $SK := \perp$.
   - $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record stored when $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$. If there is none, Game 17 sets $SK := \perp$.

   (Note that pw^* and pk^* in these two records has to match each other.) After that, Game 17 checks if $K^* = \mathsf{Decap}(sk^*, c^*)$, and computes SK using K^* or sets $SK := \perp$ accordingly. [changed in Games 11, 12, 13]

See Figure 6 for a summary of Game 17.

Comparison with the ideal world. We now claim that Game 17 is identical to the ideal world, except that the challenger in Game 17 is split into the $\mathcal{F}_{\text{PAKE-1EA}}$ functionality and the simulator $\mathcal{S}$ in the ideal world (which, of course, does not make a difference in $\mathcal{Z}$'s view).

item to be simulated	case	Game 17 (Figure 6)	simulator (Figures 2 and 3)
P-to-P' message (s, T)		line 2	the only step
P''s session key SK' and P'-to-P message (c, τ')	$(s^*, T^*) = (s, T)$	line 4–7	step 1
	no $(\mathsf{pw}')^*$, or $(\mathsf{pw}')^* \neq \mathsf{pw}'$	line 10–13	step 2(b)(i)
	$(\mathsf{pw}')^* = \mathsf{pw}'$	line 14–25	step 2(b)(ii)
P's session key SK	$(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$	line 27–30	step 1
	no pw^*, or more than one pw^*	line 33–34	step 2(a)
	no $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record	line 38–39	step 2(b)
	$K^* \neq K$	line 43–44	step 2(c)
	$\mathsf{pw}^* \neq \mathsf{pw}$	line 35–36	step 2(d)(i)
	$\mathsf{pw}^* = \mathsf{pw} \wedge K^* = K$	line 41–42	step 2(d)(ii)
$\mathcal{A}$'s $H_2(\star, T)$ queries		line 46–51	the only step

Table 1: Comparison between Game 17 and the ideal world. The "step" in simulator refers to the step in Figures 2 and 3 under the corresponding title

$P(\mathsf{pw})$	$\mathcal{A}$	$P'(\mathsf{pw}')$
$s \leftarrow \{0,1\}^{3n}; T \leftarrow \mathbb{G}$		
	$\xrightarrow{s,T}$ $\xrightarrow{s^*,T^*}$	
		if $(s^*, T^*) = (s, T)$
		$\quad SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$
		$\quad (pk', \star) \leftarrow \mathsf{Gen}$
		$\quad (c, \star) \leftarrow \mathsf{Encap}(pk')$
		otherwise
		$\quad$ extract $(\mathsf{pw}')^*$ (see Game 15)
		$\quad$ if none or $(\mathsf{pw}')^* \neq \mathsf{pw}'$
		$\qquad SK' \leftarrow \{0,1\}^n, \tau' \leftarrow \{0,1\}^{2n}$
		$\qquad pk' \leftarrow \mathbb{G}$
		$\qquad (c, \star) \leftarrow \mathsf{Encap}(pk')$
		$\quad$ if $(\mathsf{pw}')^* = \mathsf{pw}'$
		$\qquad$ if type (1) in Game 15
		$\qquad\quad t' := H_2(\mathsf{pw}', T^*)$
		$\qquad\quad r' := s^* \oplus t'$
		$\qquad\quad R' := H_1(\mathsf{pw}', r')$
		$\qquad\quad pk' := T^*/R'$
		$\qquad$ if type (2) in Game 15
		$\qquad\quad$ retrieve $\langle \mathsf{pw}', (pk')^*, \star \rangle$ record
		$\qquad\quad pk' := (pk')^* \cdot (T^*/T)$
		$\qquad (c, K') \leftarrow \mathsf{Encap}(pk')$
		$\qquad SK' := H_{\mathsf{key}}(K')$
		$\qquad \tau' := H_{\mathsf{tag}}(\mathsf{pw}', pk', s^*, T^*, c, K')$
	$\xleftarrow{c^*,\tau^*}$ $\xleftarrow{c,\tau'}$	
if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$		
$\quad SK := SK'$		
if $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$		
$\quad SK := \perp$		
otherwise		
$\quad$ find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, s, T, c^*, K^*) = \tau^*$ query		
$\quad$ if none or more than one		
$\qquad SK := \perp$		
$\quad$ if $\mathsf{pw}^* \neq \mathsf{pw}$		
$\qquad SK := \perp$		
$\quad$ retrieve $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record		
$\quad$ if none		
$\qquad SK := \perp$		
$\quad K := \mathsf{Decap}(sk^*, c^*)$		
$\quad$ if $K^* = K$		
$\qquad SK := H_{\mathsf{key}}(K)$		
$\quad$ otherwise		
$\qquad SK := \perp$		
output SK		output SK'
on $\mathcal{A}$'s fresh $H_2(\mathsf{pw}^*, T) =: t^*$ query:		
$\quad (pk^*, sk^*) \leftarrow \mathsf{Gen}$		
$\quad r^* := s \oplus t^*$		
$\quad R^* := T/pk^*$		
$\quad H_1(\mathsf{pw}^*, r^*) := R^*$		
$\quad$ store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$		

Figure 6: Game 17 (with various aborting conditions omitted)

From Table 1 (and after going through some simple but tedious checks), we can see that Game 17 and the ideal world are exactly identical. (Various aborting conditions are omitted.) This completes the proof.

5 Security Analyses of Other Versions (I): Including (s, T) in the hash

We assume the reader is familiar with the security proof in Section 4, as the simulator and many games there will be frequently referred to or reused.

5.1 What Will Go Wrong If You Don't Hash Everything?

One might fear that every new variant of OEKE — with some inputs removed from H_{tag} — needs a new security analysis from scratch, causing this paper to be hundreds of pages long. Fortunately, most of the simulator and the game hops from the proof in Section 4 can be reused. This, in fact, should be expected: only the definition of the P'-to-P authenticator τ' changes in these variants, so the only parts of the proof that are affected are P's behavior — in particular, how to extract the password guess and when to output $\perp$ — on the P'-to-P message.

Concretely, Games 2, 11 and 13 in Section 4.2 need stronger KEM security properties, or at least a new analysis, to go through. We explained in *underlined* comments in each of them why the current arguments critically require pw, pk, (s, T) and/or c as part of the H_{tag} inputs, which we summarize in Table 2:

#	summary	what needs to be included in H_{tag}
2	extract K^* from τ^*	$\mathsf{pw}, (s, T), c$
11	reject if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query	pk
13	extract pw^* from τ^* and check against K^*	pw

Table 2: Summary of games in Section 4.2 that require pw, pk and/or c as part of H_{tag} inputs

In this section, we analyze seven variants: those with (s, T) included in H_{tag}, but at least one of pw, pk, c removed. In Section B we analyze the variants where (s, T) are removed from H_{tag}.

5.2 The "Hash pw, pk and (s, T)" Version

We start from the version where τ is defined as $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, K)$, i.e., c is removed from the hash. Table 2 suggests that only Game 2 needs to change, and the simulator and all other games can be reused in the proof.

Looking more closely, only Claim 1 in Game 2 needs a new proof: the argument after this claim also does not change if we remove c from H_{tag}. Below we state and prove the new claim.

Claim 2. *Suppose the WNM property holds for the KEM scheme. In the case where* $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$*, the probability that* P' *queries* $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, K)$ *is negligible.*

Proof. P' makes a single H_{tag} query, on $(\mathsf{pw}', pk', s, T, K')$. If $\mathsf{pw} \neq \mathsf{pw}' \vee (s^*, T^*) \neq (s, T)$, then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, K)$. Now assume $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$. This implies that $pk = pk'$ by the definitions of pk and pk', and $c^* \neq c$ by the condition of this game. We now upper-bound $\Pr[K = K']$. Recall that K is defined as $\mathsf{Decap}(sk, c^*)$, so $\Pr[K = K'] = \Pr[\mathsf{Decap}(sk, c^*) = K']$. We upper-bound this probability via a reduction $\mathcal{R}_{\mathrm{WNM}}$ to the WNM property of the KEM scheme:

Reduction $\mathcal{R}_{\mathrm{WNM}}$:

On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\mathrm{WNM}}$ uses its input pk instead of $(pk, \star) \leftarrow \mathsf{Gen}$. Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise generate $(c, K') \leftarrow \mathsf{Encap}(pk)$, and compute $SK := H_{\mathsf{key}}(K')$ and $\tau' := H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, c, K')$. Output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$.

3. On (c^*, τ^*) from $\mathcal{A}$, if $c^* = c$, abort. Otherwise output c^*.

The definition of $\mathcal{R}_{\mathrm{WNM}}$ ensures that $\mathsf{Decap}(sk, c) = K'$. If $\mathcal{A}$ sends c^* such that $\mathsf{Decap}(sk, c^*) = K'$, then $\mathcal{R}_{\mathrm{WNM}}$ wins the WNM game. Thus,

$$\Pr[\mathsf{Decap}(sk, c^*) = K'] \leq \mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}}.$$

If $\mathsf{Decap}(sk, c) \neq K'$, i.e., $K \neq K'$, then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, s, T, K)$. This yields the claim. $\square$

5.3 The "Hash pw and (s, T)" Version

We now turn to the variant where pk is further removed from H_{tag} inputs, i.e., τ is defined as $H_{\mathsf{tag}}(\mathsf{pw}, s, T, K)$. Table 2 suggests that the argument under Game 11 in Section 4.2 need to (essentially) change. (Of course, since c is also removed from H_{tag}, the change in Section 5.2 also needs to be made.) Below we repeat Game 11 and make the new argument.

Game 11: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, if there is no $\langle \mathsf{pw}, \star, \star \rangle$ record, set $SK := \perp$ (i.e., replace the brown text in Figure 5 with "$SK := \perp$").

Game 10 and Game 11 differ only if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query, but makes an $H_{\mathsf{tag}}(\mathsf{pw}, s, T, K^*)$ query resulting in τ^*, where $K^* = \mathsf{Decap}(sk, c^*)$ for a freshly generated sk. We upper-bound this probability via a reduction $\mathcal{R}_{\text{UNPRE}}$ to the UNPRE property of the KEM scheme:

Reduction $\mathcal{R}_{\text{UNPRE}}$:

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$. Send (s, T) to $\mathcal{A}$.
2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, run P''s algorithm in Game 10 to compute c, SK' and τ'. Output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$.
3. On (c^*, τ^*) from $\mathcal{A}$, if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, abort. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, s, T, K^*)$ query resulting in τ^*. If there is none or more than one, abort. Otherwise output (c^*, K^*).
4. On $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ query, if $\mathsf{pw}^* = \mathsf{pw}$, abort. Otherwise do what Game 10 specifies.

It is straightforward to see that if the “bad event” above happens, then $\mathcal{R}_{\text{UNPRE}}$ wins its UNPRE game. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{10,11} \leq \mathbf{Adv}_{\mathcal{R}_{\text{UNPRE}}}^{\text{UNPRE}}.$$

Overall, we conclude that the “hash pw and (s, T)” version is secure under the WNM and UNPRE properties of the KEM scheme.

The “hash pw, (s, T) and c” version. Note that the change in Game 2 (which requires WNM) and the change in Game 11 (which require UNPRE) are unrelated to each other. Therefore, if we define τ as $H_{\mathsf{tag}}(\mathsf{pw}, s, T, c, K)$, then only Game 11 needs to change, and the protocol is secure under the UNPRE property of the KEM scheme.

5.4 The “Hash pk and (s, T)” Version

We now consider the variant where pw, but not pk, is further removed from H_{tag} inputs, i.e., τ is defined as $H_{\mathsf{tag}}(pk, s, T, K)$. Table 2 suggests that Games 2 and 13 in Section 4.2 need to (essentially) change. In fact the simulator also needs to change a bit, which we start from.

5.4.1 Changes to the Simulator

The only changes to the simulator $\mathcal{S}$ appear when $\mathcal{A}$ sends (c^*, τ^*) to P and $\mathcal{S}$ needs to extract a password guess pw^* from it. In the “hash everything” version (Section 4.1), τ^* is computed as $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, s, T, c^*, K^*)$, so $\mathcal{S}$ can extract pw^* directly from $\mathcal{A}$'s H_{tag} queries and *check its consistency with* pk^*, c^*, K^*

(see the last paragraph of Section 4.1.1). Here, $\tau^* = H_{\mathsf{tag}}(pk^*, s, T, K^*)$, which prevents $\mathcal{S}$ from reading off pw^* from $\mathcal{A}$'s H_{tag} queries. However, the inclusion of pk^* still allows $\mathcal{S}$ to find the corresponding $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, from which $\mathcal{S}$ can recover pw^* and check for consistency. Still, there is a negligible chance that there are multiple $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ records for the same pk^*, which we must rule out.

We show the new simulator in Figure 7. (Only the steps after receiving (c^*, τ^*) are included.)

<u>P's session key</u>

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(pk^*, s, T, K^*)$ query resulting in τ^*.
   (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (b) If there is exactly one such query, retrieve record $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$. If there is more than one such record, abort. If there is no such record, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (c) Compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* \neq K$, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (d) Send $(\mathsf{TestPwd}, sid, P, \mathsf{pw}^*)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P, SK)$ to $\mathcal{F}$.

Figure 7: UC simulator $\mathcal{S}$ for the "hash pk and (s, T)" variant of OEKE. Only the paragraph that needs significant change (shown in blue) is included

5.4.2 Changes in the Games

Change in Game 2. The following claim replaces Claim 1 in the "hash everything" version:

Claim 3. *Suppose the WNM property holds for the KEM scheme. In the case where* $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, *the probability that* P' *queries* $H_{\mathsf{tag}}(pk, s, T, K)$ *is negligible.*

Proof. P' makes a single H_{tag} query, on (pk', s^*, T^*, K'). If $(s^*, T^*) \neq (s, T)$, then of course P''s query is not $H_{\mathsf{tag}}(pk, s, T, K)$. Now assume $(s^*, T^*) = (s, T)$, which implies $\mathsf{pw} \neq \mathsf{pw}' \vee c^* \neq c$ by the condition of the game.

- If $\mathsf{pw} \neq \mathsf{pw}'$, we have

$$pk = \frac{T}{R} = \frac{T}{H_1(\mathsf{pw}, r)} = \frac{T}{H_1(\mathsf{pw}, s \oplus t)},$$
$$pk' = \frac{T}{R'} = \frac{T}{H_1(\mathsf{pw}', r')} = \frac{T}{H_1(\mathsf{pw}', s \oplus t')}.$$

  Since $\mathsf{pw} \neq \mathsf{pw}'$, $H_1(\mathsf{pw}, s \oplus t)$ and $H_1(\mathsf{pw}', s \oplus t')$ are independent of each other, so $\Pr[pk = pk'] = \Pr[H_1(\mathsf{pw}, s \oplus t) = H_1(\mathsf{pw}', s \oplus t')] \leq (q_1 + 1)/|\mathbb{G}|$.

- If $\mathsf{pw} = \mathsf{pw}' \wedge c^* \neq c$, then $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, which implies that $pk = pk'$ by the definition of pk and pk'. Then we can show that $\Pr[K = K']$ is negligible, via a reduction $\mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}}$ similar to that in the proof of Claim 2. We have $\Pr[K = K'] \leq \mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}}$.

If $pk \neq pk'$ or $K \neq K'$ then of course P''s query is not $H_{\mathsf{tag}}(pk, s, T, K)$. Overall, we have

$$\Pr[P' \text{ queries } H_{\mathsf{tag}}(pk, s, T, K)] \leq \mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}} + \frac{q_1 + 1}{|\mathbb{G}|},$$

which yields the claim. □

The remaining argument is essentially identical to that in the old Game 2.

Change in Game 13. We first state the new Game 13, and then make the argument.

Game 13: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau'))$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(pk^*, s, T, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \perp$. Otherwise there is a unique such $H_{\mathsf{tag}}(pk^*, s, T, K^*)$ query, and

- If there is more than one $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, abort.
- If there is no $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, set $SK := \perp$.
- If there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, but $\mathsf{pw}^* \neq \mathsf{pw}$, also set $SK := \perp$.
- If there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, and $\mathsf{pw}^* = \mathsf{pw}$, compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* = K$ then set $SK := H_{\mathsf{key}}(K)$, otherwise set $SK := \perp$.

If $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\star, s, T, \star)$ query resulting in τ^*, then in particular $\mathcal{A}$ makes no $H_{\mathsf{tag}}(pk, s, T, \star)$ query resulting in τ^*, so both Game 12 and Game 13 set $SK := \perp$. The probability that $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\star, s, T, \star)$ queries both resulting in τ^* is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)/2^{2n+1}$. (This part does not change from the argument under the old Game 13.) Below we assume that $\mathcal{A}$ makes a unique $H_{\mathsf{tag}}(pk^*, s, T, K^*)$ query resulting in τ^*:

- We first upper-bound the probability that there is more than one $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record for the same pk^*. Note that in each such record, $(pk^*, \star) \leftarrow \mathsf{Gen}$ is freshly generated, so the probability that there is a collision in pk^* is upper-bounded by $\binom{q_2}{2} \cdot p_{\text{guess}}(pk)$ (recall that there is a $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record for each $H_2(\mathsf{pw}^*, T)$ query).

- If there is no $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, then both Game 12 and Game 13 set $SK := \perp$.

- Next, suppose there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, but $\mathsf{pw}^* \neq \mathsf{pw}$. If there is no $\langle \mathsf{pw}, pk, sk \rangle$ record, then both Game 12 and Game 13 set $SK := \perp$. If there is a $\langle \mathsf{pw}, pk, sk \rangle$ record, then $(pk, \star) \leftarrow \mathsf{Gen}$ is freshly generated, so $\Pr[pk = pk^*] \leq q_2 \cdot p_{\text{guess}}(pk)$; and if $pk \neq pk^*$, then $\mathcal{A}$ does not make an $H_{\mathsf{tag}}(\mathsf{pw}, s, T, pk)$ query resulting in τ^* (because there is only one H_{tag} query resulting in τ^*, and it is on pk^*), so again both Game 12 and Game 13 set $SK := \perp$.

- Finally, if there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, and $\mathsf{pw}^* = \mathsf{pw}$, then Game 13 just runs P's algorithm in Game 12, so these two games are identical.

In sum, we have

$$\mathbf{Dist}_{\mathcal{Z}}^{12,13} \leq \frac{q_2(q_2+1)}{2} \cdot p_{\text{guess}}(pk) + \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}}-1)}{2^{2n+1}}.$$

Overall, we conclude that the "hash pk and (s, T)" version is secure under the WNM property of the KEM scheme.

The "hash pk, (s, T) and c" version. If we define τ as $H_{\mathsf{tag}}(pk, s, T, c, K)$, then the second bullet of Claim 6 is not needed, and the protocol can be proven secure without the WNM property of the KEM scheme (i.e., under the same assumptions as in the "hash everything" version).

5.5 The "Only Hash (s, T)" Version

Finally, we consider the variant where both pw and pk (as well as c) are removed from H_{tag} inputs, i.e., τ is defined as $H_{\mathsf{tag}}(s, T, K)$. This is *not* merely a combination of what we did while removing pw and while removing pk (Sections 5.3 and 5.4), and requires a new analysis. Again we will start from the changes to the simulator.

5.5.1 Changes to the Simulator

When pw is removed from H_{tag} inputs in Section 5.4, the simulator needs to change how it extracts $\mathcal{A}$'s password guess pw^*: it cannot read off pw^* from $\mathcal{A}$'s H_{tag} queries, and instead needs to read off pk^* and then fetch the $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record. Here, neither pw^* nor pk^* can be read off from $\mathcal{A}$'s H_{tag} queries, and a new extraction strategy is needed. $\mathcal{S}$ can only read off K^*, so it has to go over *all* $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ records and check which one is consistent with K^* and c^*. The new simulator is shown in Figure 8.

P's session key

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(s, T, K^*)$ query resulting in τ^*.
   (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (b) If there is exactly one such query, retrieve all records in the form of $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$. For each such record, compute $K := \mathsf{Decap}(sk^*, c^*)$ and check if $K^* = K$. If there is more than one such record, abort. If there is no such record, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (c) If there is exactly one record $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ with $\mathsf{Decap}(sk^*, c^*) = K^*$, send $(\mathsf{TestPwd}, sid, P, \mathsf{pw}^*)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P, SK)$ to $\mathcal{F}$.

Figure 8: UC simulator $\mathcal{S}$ for the "only hash (s, T)" variant of OEKE. Only the paragraph that needs significant change (shown in blue) is included

5.5.2 Changes in the Games

Change in Game 2. The following claim replaces Claim 1 in the "hash everything" version:

Claim 4. *Suppose the WNM and KBIND properties hold for the KEM scheme. In the case where* $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$*, the probability that* P' *queries* $H_{\mathsf{tag}}(s, T, K)$ *is negligible.*

Proof. P' makes a single H_{tag} query, on (s^*, T^*, K'). If $(s^*, T^*) \neq (s, T)$, then of course P''s query is not $H_{\mathsf{tag}}(s, T, K)$. Now assume $(s^*, T^*) = (s, T)$, which implies $\mathsf{pw} \neq \mathsf{pw}' \vee c^* \neq c$ by the condition of the game.

- If $\mathsf{pw} \neq \mathsf{pw}'$, by the proof of Claim 2, pk and pk' are independent of each other (where pk is honestly generated and pk' is uniformly random). This suggests that we should upper-bound $\Pr[K = K'] = \Pr[\mathsf{Decap}(sk, c^*) = K']$ via a reduction $\mathcal{R}_{\text{KBIND}}$ to the KBIND property of the KEM scheme:

Reduction $\mathcal{R}_{\text{KBIND}}$:

Sample $j \leftarrow [q_2+1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk, pk', c, K'),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{KBIND}}$ uses its input pk instead of $(pk, \star) \leftarrow \mathsf{Gen}$. Send (s, T) to $\mathcal{A}$.
2. On the j-th $H_2(\mathsf{pw}^*, T) =: t'$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\text{KBIND}}$ itself on behalf of P' — see step 3 below), compute

$$r' := s \oplus t', \quad R' := T/pk'.$$

(pk' is part of $\mathcal{R}_{\text{KBIND}}$'s input.) If $H_1(\mathsf{pw}^*, r')$ has already been defined and it is not R', abort. Otherwise program $H_1(\mathsf{pw}^*, r') := R'$. // similar to how H_1 is programmed in the OW reduction in Section 4.2, Game 6

3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise
   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.
   (b) Compute $SK := H_{\mathsf{key}}(K')$ and $\tau' := H_{\mathsf{tag}}(s, T, K')$. Output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c and K' are part of $\mathcal{R}_{\text{KBIND}}$'s input.)
4. On (c^*, τ^*) from $\mathcal{A}$, output c^*.

Assuming $\mathsf{Decap}(sk, c^*) = K'$ and $\mathcal{R}_{\text{KBIND}}$'s guess of index j is correct, the only difference between Game 1/2 and $\mathcal{R}_{\text{KBIND}}$'s simulation (until $\mathcal{A}$ sends c^*) is that $\mathcal{R}_{\text{KBIND}}$ uses its input pk' to program $H_1(\mathsf{pw}', r')$, and then uses its inputs $(c, K') \leftarrow \mathsf{Encap}(pk')$ as the P'-to-P message and compute SK and τ'. Since (c, K') is also computed as $\mathsf{Encap}(pk')$ in Game 1/2, $\mathcal{R}_{\text{KBIND}}$ simulates Game 1/2 perfectly. Furthermore, $\mathcal{R}_{\text{KBIND}}$ wins the KBIND game. We have

$$\Pr[K = K' \mid \mathsf{pw} \neq \mathsf{pw}'] \leq \mathbf{Adv}^{\text{WNM}}_{\mathcal{R}_{\text{WNM}}}.$$

- If $\mathsf{pw} = \mathsf{pw}' \wedge c^* \neq c$, then the argument in the proof of Claim 2 still holds, and $\Pr[K = K' \mid \mathsf{pw} = \mathsf{pw}' \wedge c^* \neq c] \leq (q_2+1)\mathbf{Adv}_{\mathcal{R}_{\text{KBIND}}}$.

If $K \neq K'$ then of course P''s query is not $H_{\mathsf{tag}}(s, T, K)$. Overall, we have

$$\Pr[P' \text{ queries } H_{\mathsf{tag}}(s, T, K)] = \Pr[K = K'] \leq \mathbf{Adv}^{\text{WNM}}_{\mathcal{R}_{\text{WNM}}} + (q_2+1)\mathbf{Adv}^{\text{KBIND}}_{\mathcal{R}_{\text{KBIND}}},$$

which yields the claim. □

The remaining argument is essentially identical to that in the old Game 2.

Change in Game 11. Game 10 and Game 11 differ only if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query, but makes an $H_{\mathsf{tag}}(s, T, K^*)$ query resulting in τ^*, where $K^* = \mathsf{Decap}(sk, c^*)$ for a freshly generated sk. By a reduction $\mathcal{R}_{\mathrm{UNPRE}}$ that is essentially identical to the one in Section 5.3, we have

$$\mathbf{Dist}_{\mathcal{Z}}^{10,11} \leq \mathbf{Adv}_{\mathcal{R}_{\mathrm{UNPRE}}}^{\mathrm{UNPRE}}.$$

Change in Game 13. We first state the new Game 13, and then make the argument.

Game 13: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau'))$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(s, T, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \perp$. Otherwise there is a unique such $H_{\mathsf{tag}}(s, T, K^*)$ query, and

- For each $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, compute $K := \mathsf{Decap}(sk^*, c^*)$ and check if $K^* = K$. If there is more than one such record, abort.
- If there is no such $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, set $SK := \perp$.
- If there is a unique such $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, but $\mathsf{pw}^* \neq \mathsf{pw}$, also set $SK := \perp$.
- If there is a unique such $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, and $\mathsf{pw}^* = \mathsf{pw}$, set $SK := H_{\mathsf{key}}(K)$.

If $\mathcal{A}$ makes no $H_{\mathsf{tag}}(s, T, \star)$ query resulting in τ^*, then both Game 12 and Game 13 set $SK := \perp$. The probability that $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(s, T, \star)$ queries both resulting in τ^* is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)/2^{2n+1}$. (This part does not change from the argument under the old Game 13.) Below we assume that $\mathcal{A}$ makes a unique $H_{\mathsf{tag}}(s, T, K^*)$ query resulting in τ^*:

- We first upper-bound the probability that there is more than one $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record with $\mathsf{Decap}(sk^*, c^*) = K^*$. Note that in each such record, $(pk^*, sk^*) \leftarrow \mathsf{Gen}$ is freshly generated, so a straightforward reduction to the ROB property of the KEM scheme, $\mathcal{R}_{\mathrm{ROB1}}$, shows that the probability that the same c^* decapsulates to the same K^* under two freshly generated sk^* is upper-bounded by $\binom{q_2}{2} \cdot \mathbf{Adv}_{\mathcal{R}_{\mathrm{ROB1}}}^{\mathrm{ROB}}$.
- If there is no $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record with $\mathsf{Decap}(sk^*, c^*) = K^*$, then both Game 12 and Game 13 set $SK := \perp$.
- Next, suppose there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record with $\mathsf{Decap}(sk^*, c^*) = K^*$, but $\mathsf{pw}^* \neq \mathsf{pw}$. If there is no $\langle \mathsf{pw}, pk, sk \rangle$ record, then both Game 12 and Game 13 set $SK := \perp$. If there is a $\langle \mathsf{pw}, pk, sk \rangle$ record, then $(pk, sk) \leftarrow \mathsf{Gen}$ is freshly generated, so a straightforward reduction to the ROB property of the KEM scheme, $\mathcal{R}_{\mathrm{ROB2}}$, shows that the probability that the same c^* decapsulates to the same K^* under both sk and sk^* is upper-bounded by $q_2 \cdot \mathbf{Adv}_{\mathcal{R}_{\mathrm{ROB2}}}^{\mathrm{ROB2}}$. If this does not happen, then again both Game 12 and Game 13 set $SK := \perp$.

- Finally, if there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, and $\mathsf{pw}^* = \mathsf{pw}$, then both Game 12 and Game 13 set $SK := H_{\mathsf{key}}(K)$.

In sum, we have

$$\mathbf{Dist}_{\mathcal{Z}}^{12,13} \leq \frac{q_2(q_2-1)}{2} \cdot \mathbf{Adv}_{\mathcal{R}_{\mathrm{ROB1}}}^{\mathrm{ROB1}} + q_2 \cdot \mathbf{Adv}_{\mathcal{R}_{\mathrm{ROB2}}}^{\mathrm{ROB2}} + \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}}-1)}{2^{2n+1}}.$$

Overall, we conclude that the "only hash (s, T)" version is secure under the WNM, UNPRE, KBIND and ROB properties of the KEM scheme.

The "hash (s, T) and c" version. If we define τ as $H_{\mathsf{tag}}(s, T, c, K)$, then the second bullet of Claim 4 is not needed, and the protocol can be proven secure without the WNM property of the KEM scheme (but we still need KBIND, UNPRE and ROB).

References

[ABJ25] Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. Cryptology ePrint Archive, Report 2025/231, 2025.

[ABN10] Michel Abdalla, Mihir Bellare, and Gregory Neven. Robust encryption. In *TCC 2010*, pages 480–497, 2010.

[ABP15] Michel Abdalla, Fabrice Benhamouda, and David Pointcheval. Public-key encryption indistinguishable under plaintext-checkable attacks. In *PKC 2015*, pages 332–352, 2015.

[BDZ03] Feng Bao, Robert H. Deng, and Huafei Zhu. Variations of Diffie-Hellman problem. In *ICICS 2003*, pages 301–312, 2003.

[BS23] Dan Boneh and Victor Shoup. A graduate course in applied cryptography (version 0.6), 2023. https://toc.cryptobook.us/book.pdf.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS 2001*, pages 136–145, 2001.

[FOR17] Pooya Farshim, Claudio Orlandi, and Răzvan Roşie. Security of symmetric primitives under incorrect usage of keys. *ToSC*, 2017(1):449–473, 2017.

[JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.

[LLMT25] Chengyu Lin, Zeyu Liu, Peihan Miao, and Max Tromanhauser. Finding balance in unbalanced PSI: A new construction from single-server PIR. *CiC*, 2(1):27, 2025.

[MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.

A Comparison Between Our Simulator and the One in [ABJ25]

Our simulator in Section 4.1.2 is, by and large, the same as the one in [ABJ25, Fig.5 and 6], although we completely rewrite it to improve readability. We first summarize some of the notational differences in Table 3.

	our notation	notation in [ABJ25]	comment
protocol parties	P, P'	A, B	easy to confuse with adversary $\mathcal{A}$
simulated P-to-P' message	(s, T)	$(\overline{s}, \overline{T})$	
adversarial P-to-P' message	(s^*, T^*)	(s, T)	
$\mathcal{A}$'s guess of P''s password	$(\mathsf{pw}')^*$	pw^*	
KEM public key corresponding to correct password guess on P'	pk'	pk^*	
simulated P'-to-P message	(c, τ')	(c, tag)	
adversarial P'-to-P message	(c^*, τ^*)	(c, tag)	same (c, tag) for different things
$\mathcal{A}$'s guess of P's password	pw^*	pw^*	same pw^* for different things

Table 3: Comparison of notations between our simulator and the one in [ABJ25]

The simulator in [ABJ25] has more bookkeeping and aborts more frequently. In particular, we completely remove their record forwS, and when detecting whether an adversarial P-to-P' message contains more than one password guess (causing the simulator to abort), we only check the relevant H_2 queries, whereas they check *all* H_2 queries — including those that are completely unrelated to the attack in the P-to-P' direction. For a more detailed explanation, see the comment after Game 15 in Section 4.2.

The only (more or less) essential difference lies in the red text in Figure 2: in the case where the adversary makes an incorrect password guess on P', we compute c from a uniformly random KEM public key pk', whereas they compute c from an honestly generated pk'. While both methods work, our method simplifies the reduction; this is further explained in the comment after Game 18 in Section 4.2, item (2).

B Security Analyses of Other Versions (II): Removing (s, T) from the Hash

B.1 The "Hash pw, pk and c" Version

We start from the version where τ is defined as $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K)$, i.e., only (s, T) are removed from the hash compared with the "hash everything" version.

Fortunately, in this variant only the proof of Claim 2 in Game 2 needs to (essentially) change, and the simulator and all other arguments can be reused. This is because Claim 2 still holds (except that Claim 2 holds perfectly in the "hash everything" version, whereas here we need to make a statistical argument), and by Table 2, Game 2 is the only game in the entire security proof where we need (s, T) to be included in H_{tag} inputs.

Claim 5. *In the case where* $\mathsf{pw} \neq \mathsf{pw}' \vee ((s^*, T^*) = (s, T) \wedge c^* \neq c)$*, the probability that* P' *queries* $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ *is negligible.*

Proof. P' makes a single H_{tag} query, on $(\mathsf{pw}', pk', c, K')$. If $\mathsf{pw} \neq \mathsf{pw}' \vee c^* \neq c$, then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$. Now assume $\mathsf{pw} = \mathsf{pw}' \wedge c^* = c$. This implies that $(s^*, T^*) \neq (s, T)$ by the condition of this game. We have

$$pk = \frac{T}{R} = \frac{T}{H_1(\mathsf{pw}, r)} = \frac{T}{H_1(\mathsf{pw}, s \oplus t)} = \frac{T}{H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T))},$$
$$pk' = \frac{T^*}{R'} = \frac{T^*}{H_1(\mathsf{pw}, r')} = \frac{T^*}{H_1(\mathsf{pw}, s^* \oplus t')} = \frac{T^*}{H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))}.$$

There are three cases:

- If $s^* \neq s \wedge T^* = T$, then $H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T))$ and $H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))$ are independent of each other, so $\Pr[pk = pk'] = \Pr[H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T)) = H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))] \leq (q_1 + 1)/|\mathbb{G}|$.
- If $T^* \neq T \wedge s \oplus H_2(\mathsf{pw}, T) = s^* \oplus H_2(\mathsf{pw}, T^*)$, then $H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T)) = H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))$, so $pk \neq pk'$.
- If $T^* \neq T \wedge s \oplus H_2(\mathsf{pw}, T) \neq s^* \oplus H_2(\mathsf{pw}, T^*)$, then $H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T))$ and $H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))$ are independent of each other, so $\Pr[pk = pk'] = \Pr[H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T))/H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*)) = T/T^*] \leq (q_1 + 1)/|\mathbb{G}|$.

If $pk \neq pk'$ then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$. Overall, we have

$$\Pr[P' \text{ queries } H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)] \leq \Pr[pk = pk'] \leq \frac{q_1 + 1}{|\mathbb{G}|},$$

which yields the claim. □

B.2 The "Hash pw" Version

We now turn to the variant where pk is further removed from H_{tag} inputs, i.e., τ is defined as $H_{\mathsf{tag}}(\mathsf{pw}, K)$. Table 2 suggests that Games 2 and 11 in Section 4.2 need to (essentially) change. In fact the simulator also needs to change quite a bit, which we will start from.

B.2.1 Changes to the Simulator

The only changes to the simulator $\mathcal{S}$ appear when $\mathcal{A}$ sends (c^*, τ^*) to P and $\mathcal{S}$ needs to extract a password guess pw^* from it. In the "hash everything" version (Section 4.1), τ^* is computed as $H_{\mathsf{tag}}(\mathsf{pw}^*, pk^*, c^*, K^*)$, so $\mathcal{S}$ can extract pw^* from $\mathcal{A}$'s H_{tag} queries and *check its consistency with* pk^*, c^*, K^* (see the last paragraph of Section 4.1.1). Here, $\tau^* = H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$, but the inclusion of pw^* still allows $\mathcal{S}$ to find the corresponding $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record and check for consistency.

However, a more sinister issue emerges when we remove pk^* from H_{tag}: In the case where the two parties' passwords match, $\mathcal{A}$ can modify c and force the two parties' session keys to be the same — without making *any* H_{tag} query! The easiest way to see this is to work out a concrete version for OEKE based on the Diffie-Hellman KEM, which we show in Figure 9.

$$
\begin{array}{lcccl}
P(\mathsf{pw}) & & \mathcal{A} & & P'(\mathsf{pw}) \\
 & \xrightarrow{s,\,T} & & & \\
 & & X := \mathsf{2F}^{-1}(\mathsf{pw}, (s, T)) & & \\
 & & (s^*, T^*) := \mathsf{2F}(\mathsf{pw}, X^2) & & \\
 & & & \xrightarrow{s^*,\,T^*} & \\
 & \xleftarrow{Y^2,\,\tau'} & & \xleftarrow{Y,\,\tau'} & \\
\text{output} & & & & \text{output} \\
H_{\mathsf{key}}((Y^2)^x) = H_{\mathsf{key}}(g^{2xy}) & & & & H_{\mathsf{key}}((X^2)^y) = H_{\mathsf{key}}(g^{2xy})
\end{array}
$$

Figure 9: New attack on the "hash pw" version

In this attack, the two honest parties' passwords pw are the same. Assuming $\mathcal{A}$ knows it, it can decrypt P's message to recover $X = g^x$, and then re-encrypt X^2 under the same pw; this causes P' to output $SK' = H_{\mathsf{key}}(g^{2xy})$ and send $(Y = g^y, \tau' = H_{\mathsf{tag}}(\mathsf{pw}, g^{2xy}))$. Then $\mathcal{A}$ sends (Y^2, τ') to P, causing P's check to pass and P will also output $H_{\mathsf{key}}(g^{2xy})$. (This is not an issue if pk is included in H_{tag}, since P would compute $\tau = H_{\mathsf{tag}}(\ldots, Y^2, \ldots) \neq H_{\mathsf{tag}}(\ldots, Y, \ldots) = \tau'$ and its check would fail.)

In fact, it is a stretch to call this an attack, since the only thing $\mathcal{A}$ does is to modify some protocol messages and cause P and P' to output the same session key — which $\mathcal{A}$ *cannot* compute. Still, this scenario is not covered by the previous simulator. It should be simulated as follows: When $\mathcal{A}$ decrypts (s, T) under pw, $\mathcal{S}$ records $\langle \mathsf{pw}, pk, sk \rangle$ (although there might be a large number of such records, and $\mathcal{S}$ does not know which is the "right" one at this point). Then when $\mathcal{A}$ sends (s^*, T^*) to P', $\mathcal{S}$ extracts pw, which allows $\mathcal{S}$ to retrieve the "right" record $\langle \mathsf{pw}, pk, sk \rangle$; $\mathcal{S}$ also computes K' while simulating the P' side. Later, when $\mathcal{A}$ sends (c^*, τ'), $\mathcal{S}$ can tell if $\mathcal{A}$ is performing the aforementioned attack by checking if $\mathsf{Decap}(sk, c^*) = K'$.

We show the new simulator in Figure 10. (Only the steps after receiving (c^*, τ^*) are included.)

P's session key

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. If $(s^*, T^*) \neq (s, T) \wedge \tau^* = \tau'$, and there has been a $(\mathsf{TestPwd}, sid, P, \mathsf{pw}')$ command to $\mathcal{F}$ resulting in "correct guess" (i.e., $\mathcal{S}$ previously entered step 2(b)(ii) in Figure 3), there must be a K' defined at that time.
   (a) Retrieve record $\langle \mathsf{pw}', \star, sk \rangle$. If there is no such record, go to step 3.
   (b) Compute $K := \mathsf{Decap}(sk, c^*)$. If $K \neq K'$, go to step 3.
   (c) Send $(\mathsf{TestPwd}, sid, P, \mathsf{pw}')$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P, SK)$ to $\mathcal{F}$.
3. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$ query resulting in τ^*.
   (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (b) If there is exactly one such query, retrieve record $\langle \mathsf{pw}^*, \star, sk^* \rangle$. If there is no such record, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (c) Compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* \neq K$, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (d) Send $(\mathsf{TestPwd}, sid, P, \mathsf{pw}^*)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P, SK)$ to $\mathcal{F}$.

Figure 10: UC simulator $\mathcal{S}$ for the "hash pk" variant of OEKE. Only the paragraph that needs significant change (shown in blue) is included

B.2.2 Changes in Game Hops

Note that Claim 1 in Game 2 does not hold anymore, exactly due to what the adversary can do in Figure 9 (P does not reject even though $\mathcal{A}$ makes no H_{tag} query whatsoever). As such, Game 2 needs to change essentially. Since Game 2 is almost at the beginning of the sequence of games, this affects some subsequent games and we need to proceed carefully. (Most of it is clerical work verifying that the previous argument still goes through; the parts that need to change essentially are shown in blue.) We start from describing the new Game 2.

Game 2: In the case where $\mathsf{pw} \neq \mathsf{pw}' \vee ((s^*, T^*) = (s, T) \wedge c^* \neq c)$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \bot$. If there is a unique such query, do the following:

- If $K^* = K$, set $SK := H_{\mathsf{key}}(K)$ (i.e., there is no change from Game 1).
- If $K^* \neq K$, set $SK := \bot$.

Furthermore, in the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, do the same as above, except that we now find $\mathcal{A}$ *or P'*'s $H_{\mathsf{tag}}(\mathsf{pw}, K^*)$ query resulting in τ^*.

The difference between the new Game 2 and the old one is that we now have two separate cases:

1. $\mathsf{pw} \neq \mathsf{pw}' \vee ((s^*, T^*) = (s, T) \wedge c^* \neq c)$;
2. $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$.

These two cases combined yield $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, which is identical to the case in the old Game 2. In case (1) we proceed as in the old Game 2, whereas in case (2) we cannot due to the "attack" in Figure 9; instead, in case (2) we need to include P''s H_{tag} query as if it were $\mathcal{A}$'s.

Claim 6. *Suppose the WNM property holds for the KEM scheme. In the first case above, the probability that P' queries $H_{\mathsf{tag}}(\mathsf{pw}, K)$ is negligible.*

The proof of this claim is essentially part of the proof of Claim 2. P' makes a single H_{tag} query, on (pw', K'). There are two cases:

- If $\mathsf{pw} \neq \mathsf{pw}'$, then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, K)$.
- If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* \neq c$, then the argument in the proof of Claim 2 still goes through, so $\Pr[K = K'] \leq \mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}}$.

This yields the claim. The remaining argument is essentially identical to that in the old Game 2.

Games 3–4: These two games deal with the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$. This falls into case (1) in Game 2, and in this case there is no difference between the old and new versions of Game 2. As such, the reductions still go through: they abort if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ does not hold, and if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, they can still simulate the games by checking $\mathcal{A}$'s H_{tag} queries. Crucially, the reductions do not need to check P''s H_{tag} query (which they cannot do), since in this case we have defined Game 2 so that it only checks $\mathcal{A}$'s H_{tag} queries.

Game 5: This game involves a reduction to the UNI-PK property of the KEM scheme. The reduction works essentially because sk' is not used anywhere in the game (so the reduction can embed its challenge as pk' without knowing sk'), and this fact still holds even though we need to additionally check P''s H_{tag} queries in some cases now. So, this reduction still goes through.

Games 6–8: These three games deal with the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$. This again falls into case (1) in Game 2, so the reductions in Games 6 and 7 still go through; Game 8 involves a statistical argument which does not need to change either.

Games 9–10: These two games involve statistical arguments only, and they have nothing to do with whether the games need to check P''s H_{tag} queries.

Game 11: The argument for this game needs to change essentially (see Table 2). Game 10 and Game 11 differ if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query, but $\mathcal{A}$ *or* P' makes an $H_{\mathsf{tag}}(\mathsf{pw}, K^*)$ query resulting in τ^*, where $K^* = \mathsf{Decap}(sk, c^*)$ for a freshly generated sk. A straightforward reduction to the UNPRE property of the KEM scheme, $\mathcal{R}_{\text{UNPRE1}}$, shows that

$$\mathbf{Dist}_{\mathcal{Z}}^{10,11} \leq \mathbf{Adv}_{\mathcal{R}_{\text{UNPRE1}}}^{\text{UNPRE}}.$$

Game 12: This game should read as follows now: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$, do what's in the $\mathsf{pw} \neq \mathsf{pw}' \vee ((s^*, T^*) = (s, T) \wedge c^* \neq c)$ case. That is, first find the $\langle \mathsf{pw}, \star, \star \rangle$ record (as per Game 11), then find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \perp$. If there is a unique such query, do the following:

- If $K^* = K$, set $SK := H_{\mathsf{key}}(K)$.
- If $K^* \neq K$, set $SK := \perp$.

In Game 11 SK is outright set to $\perp$, whereas here it checks $\mathcal{A}$'s H_{tag} queries *but does not check P''s H_{tag} query.* Therefore, this game hop has nothing to do with what's different in the new Game 2, and the previous argument still goes through.

Game 13: This game should now extract pw^* from $\mathcal{A}$ *or* P''s H_{tag} query, based on the cases in Game 2 and Game 12. For completeness, we include a full description of the game.

In the case where $\mathsf{pw} \neq \mathsf{pw}' \vee ((s^*, T^*) = (s, T) \wedge (c^* \neq c \vee (c^* = c \wedge \tau^* \neq \tau')))$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \perp$. Otherwise there is a unique such $H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$ query, and

- If $\mathsf{pw}^* \neq \mathsf{pw}$, set $SK := \perp$.
- If $\mathsf{pw}^* = \mathsf{pw}$, run the remaining part of P''s algorithm except that we now use pw^* instead of pw — that is, if there is no $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record then set $SK := \perp$; otherwise compute $K := \mathsf{Decap}(sk^*, c^*)$, and if $K^* = K$ then set $SK := H_{\mathsf{key}}(K)$, otherwise set $SK := \perp$.

Furthermore, in the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, do the same as above, except that we now find $\mathcal{A}$ *or* P''s $H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$ query resulting in τ^*.

The previous statistical argument does not really change.

Games 14–16: The statistical arguments for these three games do not need to change.

New game 1: Now that $(\mathsf{pw}')^*$ — the password guess extracted from (s^*, T^*) — is defined, we can finally add (part of) what's in step 2 of Figure 10. Concretely, we change the game as follows:

In the case where $(s^*, T^*) \neq (s, T) \wedge (\mathsf{pw}')^* = \mathsf{pw} = \mathsf{pw}' \wedge \tau^* = \tau'$, there is a record $\langle \mathsf{pw}, \star, sk \rangle$, and $\mathsf{Decap}(sk, c^*) = K'$, set $SK := H_{\mathsf{key}}(K')$. (In some sense this is the most crucial new game, as it covers the scenario in Figure 9 — which is why we need to make all these changes to begin with. It also corresponds to step 2(c)(ii) in Figure 10. We will deal with step 2(c)(i) later.)

Since $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, Game 16 finds $\mathcal{A}$ or P''s H_{tag} query resulting in τ^* (see the description of Game 13). P' makes a singe query to H_{tag}, on input (pw', K'); and the query result is $\tau' = \tau^*$. Therefore, Game 16 finds this query, and does the following (again see the description of Game 13):

1. Check if $\mathsf{pw}' = \mathsf{pw}$, which is indeed the case.
2. Find the $\langle \mathsf{pw}, pk, sk \rangle$ record.
3. Compute $K := \mathsf{Decap}(sk, c^*)$ and check if $K = K'$, which is again the case by the condition of this game.
4. Set $SK := H_{\mathsf{key}}(K)(= H_{\mathsf{key}}(K'))$.

This entire process in Game 16 eventually results in setting $SK := H_{\mathsf{key}}(K')$, which exactly matches the new game. Thus, there is no difference. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{16,\mathrm{new1}} = 0.$$

New game 2: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, we find $\mathcal{A}$ *or* P''s $H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$ query resulting in τ^* (except that in new game 1's condition, new game 1 already shortcuts it to $SK := H_{\mathsf{key}}(K')$ without trying to find any H_{tag} query). In this game, we do not check P''s H_{tag} query anymore, and only check $\mathcal{A}$'s H_{tag} query.

There are the following cases:

- If $\tau^* \neq \tau'$, then of course P''s H_{tag} query does not result in τ^* (it results in τ'), so we can safely drop P''s H_{tag} query.
- If $(\mathsf{pw}')^* \neq \mathsf{pw}' \wedge \tau^* = \tau'$, then P''s $H_{\mathsf{tag}}(\mathsf{pw}', K')$ query results in τ^*, and we need to argue that this query can be dropped from the check.
  - If $\mathcal{A}$ queries $H_{\mathsf{tag}}(\mathsf{pw}', K')$, then new game 2 still finds this query, so new game 1 and new game 2 are identical.

– If $\mathcal{A}$ does not query $H_{\mathsf{tag}}(\mathsf{pw}', K')$, then new game 2 sets $SK := \perp$. We argue that new game 1 also sets $SK := \perp$ except with negligible probability. New game 1 finds P''s $H_{\mathsf{tag}}(\mathsf{pw}', K')$ query, and sets $SK := \perp$ unless $\mathsf{Decap}(sk, c^*) = K'$. But note that sk is freshly generated, so a straightforward reduction to the UNPRE property of the KEM scheme, $\mathcal{R}_{\text{UNPRE2}}$, shows that $\mathsf{Decap}(sk, c^*) = K'$ happens with negligible probability (cf. the reduction in Game 11).

- If $(\mathsf{pw}')^* = \mathsf{pw}' \wedge \tau^* = \tau'$, but there is no $\langle \mathsf{pw}, \star, sk \rangle$ record, then both new game 1 and new game 2 outright sets $SK := \perp$, regardless of H_{tag} queries by $\mathcal{A}$ or P'. Thus, the two games are identical.
- If $(\mathsf{pw}')^* = \mathsf{pw}' \wedge \tau^* = \tau'$, but $\mathsf{Decap}(sk, c^*) \neq K'$, then new game 1 finds P''s $H_{\mathsf{tag}}(\mathsf{pw}', K')$ query and then sets $SK := \perp$ because the check $\mathsf{Decap}(sk, c^*) \stackrel{?}{=} K'$ does not pass, and new game 2 also sets $SK := \perp$ because it either fails to find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}', K')$ query, or it finds such a query but then sets $SK := \perp$ because the check $\mathsf{Decap}(sk, c^*) \stackrel{?}{=} K'$ does not pass. Thus, the two games are identical.

The remaining case is that $(\mathsf{pw}')^* = \mathsf{pw}' \wedge \tau^* = \tau'$, there is a $\langle \mathsf{pw}, \star, sk \rangle$ record, and $\mathsf{Decap}(sk, c^*) = K'$. This is exactly the condition in new game 1, and new game 1 outright sets $SK := H_{\mathsf{key}}(K')$ without trying to find any H_{tag} query (by $\mathcal{A}$ or P'), so the game change does not affect this case. Overall, we conclude that

$$\mathbf{Dist}_{\mathcal{Z}}^{\text{new1,new2}} \leq \mathbf{Adv}_{\mathcal{R}_{\text{UNPRE2}}}^{\text{UNPRE}}.$$

Game 17: Note that after new game 2, we never try to find P''s H_{tag} query anymore: in some cases we try to find $\mathcal{A}$'s H_{tag} query, and in other cases we outright set SK to be a certain value or $\perp$. Therefore, the reduction still goes through (it can simulate the P side honestly).[18]

New game 3: In the case where $(s^*, T^*) \neq (s, T) \wedge (\mathsf{pw}')^* = \mathsf{pw}' \neq \mathsf{pw} \wedge \tau^* = \tau'$, there is a record $\langle \mathsf{pw}', \star, sk \rangle$, and $\mathsf{Decap}(sk, c^*) = K'$, set $SK := \perp$. (This corresponds to step 2(c)(i) in Figure 10.)

In this case Game 17 tries to find $\mathcal{A}$'s (unique) $H_{\mathsf{tag}}(\mathsf{pw}^*, K^*)$ query resulting in τ^*, and checks if $\mathsf{pw}^* = \mathsf{pw}$ (see Game 13). But $\tau^* = \tau' = H_{\mathsf{tag}}(\mathsf{pw}', K')$, so $\mathsf{pw}^* = \mathsf{pw}' \neq \mathsf{pw}$ and the check never passes — that is, both Game 17 and new game 3 set $SK := \perp$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{17,\text{new3}} = 0.$$

Summary. We have made the following essential changes to the games:

[18]New game 1 and new game 2 — which drop P''s H_{tag} query from the check — have to be inserted before Game 17, since the reduction in Game 17 cannot check P''s H_{tag} query (the query is $H_{\mathsf{tag}}(\mathsf{pw}', K')$, and the reduction does not know K').

- In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, Game 2 tries to find P''s H_{tag} query resulting in τ^* (in addition to $\mathcal{A}$'s query).
- In the case where $(s^*, T^*) \neq (s, T) \wedge (\mathsf{pw}')^* = \mathsf{pw} = \mathsf{pw}' \wedge \tau^* = \tau'$, there is a record $\langle \mathsf{pw}, \star, sk \rangle$, and $\mathsf{Decap}(sk, c^*) = K'$, new game 1 outright sets $SK := H_{\mathsf{key}}(K')$ (without trying to find any H_{tag} query by $\mathcal{A}$ or P').
- In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, new game 2 drops the check on P''s H_{tag} query introduced in Game 2 (except in new game 1's condition, where all checks on H_{tag} queries have already been dropped).
- In the case where $(s^*, T^*) \neq (s, T) \wedge (\mathsf{pw}')^* = \mathsf{pw}' \neq \mathsf{pw} \wedge \tau^* = \tau'$, there is a record $\langle \mathsf{pw}', \star, sk \rangle$, and $\mathsf{Decap}(sk, c^*) = K'$, new game 3 outright sets $SK := \perp$.

Overall, the net changes are that in the case where $(s^*, T^*) \neq (s, T) \wedge (\mathsf{pw}')^* = \mathsf{pw}' \wedge \tau^* = \tau'$, there is a record $\langle \mathsf{pw}', \star, sk \rangle$, and $\mathsf{Decap}(sk, c^*) = K'$,

- If $(\mathsf{pw}')^* = \mathsf{pw}$, then we set $SK := H_{\mathsf{key}}(K')$.
- If $(\mathsf{pw}')^* \neq \mathsf{pw}$, then we set $SK := \perp$.

This exactly matches step 2 of Figure 10, so new game 3 is identical to the ideal world. This completes the proof.

B.3 The "Hash pk" Version

We now consider the variant where pw, but not pk, is further removed from H_{tag} inputs, i.e., τ is defined as $H_{\mathsf{tag}}(pk, K)$. Table 2 suggests that Games 2 and 13 in Section 4.2 need to (essentially) change; however, we will (again) start from the changes to the simulator.

B.3.1 Changes to the Simulator

Similar to Section B.2, once we remove pk from H_{tag}, $\mathcal{A}$ can force the two parties' session keys to be the same without performing a "real" attack. However, this time the two parties' passwords are different (and $\mathcal{A}$ needs to know both of them), and $\mathcal{A}$ should forward (c, τ') from P' without modifications. We show this attack in Figure 11.

$$
\begin{array}{lcccc}
P(\mathsf{pw}) & & \mathcal{A} & & P'(\mathsf{pw}') \\
& \xrightarrow{s, T} & & & \\
& & pk := \mathsf{2F}^{-1}(\mathsf{pw}, (s, T)) & & \\
& & (s^*, T^*) := \mathsf{2F}(\mathsf{pw}', pk) & & \\
& & & \xrightarrow{s^*, T^*} & \\
& \xleftarrow{c, \tau'} & & \xleftarrow{c, \tau'} & \\
\text{output } SK' & & & & \text{output } SK'
\end{array}
$$

Figure 11: New attack on the "hash pk" version

In this attack, the two honest parties' passwords pw and pw' may or may not be the same. Assuming $\mathcal{A}$ knows both, it can decrypt P's message to recover pk, and then re-encrypt pk under pw'; this ensures that P' will recover the same pk as on the P side, and use it to compute c, K', τ' (and eventually SK'). But then if $\mathcal{A}$ forwards (c, τ') without any modification, P will decapsulate c to the same K' (because it uses the same pk), and its output key will be the same as SK' on the P' side. (This is not an issue if pw is included in H_{tag}, since forwarding $\tau' = H_{\mathsf{tag}}(\mathsf{pw}', \dots) \neq H_{\mathsf{tag}}(\mathsf{pw}, \dots)$ to P would make P's check fail.)

This is (again) a new situation not covered by the previous simulator. It should be simulated as follows: When $\mathcal{A}$ decrypts (s, T) under pw, $\mathcal{S}$ records $\langle \mathsf{pw}, pk, sk \rangle$ (although there might be a large number of such records, and $\mathcal{S}$ does not know which is the "right" one at this point). Then when $\mathcal{A}$ sends (s^*, T^*) to P', $\mathcal{S}$ extracts pw', *which also gives a* pk'. Later, when $\mathcal{A}$ forwards (c, τ'), $\mathcal{S}$ knows that $\mathcal{A}$ is performing the aforementioned attack, and checks which $\langle \mathsf{pw}, pk, sk \rangle$ record matches pk'. In this way $\mathcal{S}$ can extract the password guess pw on the P side.

We show the new simulator in Figure 12. (Only the steps after receiving (c^*, τ^*) are included.)

P's session key

On (c^*, τ^*) from $\mathcal{F}$, simulate P's output as follows:

1. If $(s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, send $(\mathsf{NewKey}, sid, P', 0^n)$ to $\mathcal{F}$. // $\mathcal{A}$ is passive
2. If $(s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, and there has been a $(\mathsf{TestPwd}, sid, P', \mathsf{pw}')$ command to $\mathcal{F}$ resulting in "correct guess" (i.e., $\mathcal{S}$ previously entered step 2(b)(ii) in Figure 3), there must be a pk' and an SK' defined at that time. If there is more than one record in the form of $\langle \mathsf{pw}^*, pk', \star \rangle$, abort. If there is a unique such record, send $(\mathsf{TestPwd}, sid, P, \mathsf{pw}^*)$ to $\mathcal{F}$.
   (i) If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (ii) If $\mathcal{F}$ replies with "correct guess", send $(\mathsf{NewKey}, sid, P, SK')$ to $\mathcal{F}$.
3. Otherwise find $\mathcal{A}$'s $H_{\mathsf{tag}}(pk^*, K^*)$ query resulting in τ^*.
   (a) If there is no such query or more than one such query, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (b) If there is exactly one such query, retrieve record $\langle \mathsf{pw}^*, \star, sk^* \rangle$. If there is more than one such record, abort. If there is no such record, send $(\mathsf{TestPwd}, sid, P, \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (c) Compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* \neq K$, send $(\mathsf{TestPwd}, sid, P', \perp)$ and then $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
   (d) Send $(\mathsf{TestPwd}, sid, P, \mathsf{pw}^*)$ to $\mathcal{F}$.
      i. If $\mathcal{F}$ replies with "wrong guess", send $(\mathsf{NewKey}, sid, P, 0^n)$ to $\mathcal{F}$.
      ii. If $\mathcal{F}$ replies with "correct guess", compute $SK := H_{\mathsf{key}}(K)$ and send $(\mathsf{NewKey}, sid, P, SK)$ to $\mathcal{F}$.

Figure 12: UC simulator $\mathcal{S}$ for the "hash pk" variant of OEKE. Only the paragraph that needs significant change (shown in blue) is included

B.3.2 Changes in Game Hops

The changes in game hops again need to start from Game 2. The structure of the proof is very similar to Section B.2.2; however, for the sake of completeness we still include a proof on the same level of details as in Section B.2.2. (This involves copying a large chunk of arguments verbatim from Section B.2.2.)

Game 2: In the case where $\mathsf{pw} = \mathsf{pw}' \vee (s^*, T^*) = (s, T) \vee c^* \neq c$ yet $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$ (i.e., excluding the condition of Game 1), find $\mathcal{A}$'s $H_{\mathsf{tag}}(pk, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \perp$. If there is a unique such query, do the following:

- If $K^* = K$, set $SK := H_{\mathsf{key}}(K)$ (i.e., there is no change from Game 1).

- If $K^* \neq K$, set $SK := \perp$.

Furthermore, in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge c^* = c$, do the same as above, except that we now find $\mathcal{A}$ *or* P''s $H_{\mathsf{tag}}(pk, K^*)$ query resulting in τ^*.

Claim 7. *Suppose the WNM property holds for the KEM scheme. In the first case above, the probability that* P' *queries* $H_{\mathsf{tag}}(pk, K)$ *is negligible.*

P' makes a single H_{tag} query, on (pk', K'). There are the following cases:

- If $c^* \neq c$, we claim that $pk = pk' \wedge K = K'$ happens with negligible probability. Suppose $pk = pk'$. Then we can reduce the event $K = K'$ to the WNM property of the KEM scheme, via a reduction $\mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}}$ similar to that in the proof of Claim 2. We have $\Pr[pk = pk' \wedge K = K'] \leq \mathbf{Adv}^{\mathrm{WNM}}_{\mathcal{R}_{\mathrm{WNM}}}$.
- If $c^* = c$, then $\mathsf{pw} = \mathsf{pw}' \vee (s^*, T^*) = (s, T)$ (by the condition of this game) yet $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$ (otherwise we fall into the condition of Game 1).
  - If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T)$, then the argument in the proof of Claim 1 still goes through, so $\Pr[pk = pk'] \leq (q_1 + 1)/|\mathbb{G}|$.
  - If $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, this case is not covered by previous proofs and needs a new argument. In this case we have

$$pk = \frac{T}{R} = \frac{T}{H_1(\mathsf{pw}, r)} = \frac{T}{H_1(\mathsf{pw}, s \oplus t)},$$
$$pk' = \frac{T}{R'} = \frac{T}{H_1(\mathsf{pw}', r')} = \frac{T}{H_1(\mathsf{pw}', s \oplus t')}.$$

    Since $\mathsf{pw} \neq \mathsf{pw}'$, $H_1(\mathsf{pw}, s \oplus t)$ and $H_1(\mathsf{pw}', s \oplus t')$ are independent of each other, so $\Pr[pk = pk'] = \Pr[H_1(\mathsf{pw}, s \oplus t) = H_1(\mathsf{pw}', s \oplus t')] \leq (q_1 + 1)/|\mathbb{G}|$.

This yields the claim. The remaining argument is essentially identical to that in the old Game 2.

Games 3–4: These two games deal with the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$. This falls into the first case of Game 2, and in this case there is no difference between the old and new versions of Game 2. As such, the reductions still go through: they abort if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ does not hold, and if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, they can still simulate the games by checking $\mathcal{A}$'s H_{tag} queries. Crucially, the reductions do not need to check P''s H_{tag} query (which they cannot do), since in this case we have defined Game 2 so that it only checks $\mathcal{A}$'s H_{tag} queries.

Game 5: This game involves a reduction to the UNI-PK property of the KEM scheme. The reduction works essentially because sk' is not used anywhere in the game (so the reduction can embed its challenge as pk' without knowing sk'), and this fact still holds even though we need to additionally check P''s H_{tag} queries in some cases now. So, this reduction still goes through.

Games 6–8: These three games deal with the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$. This again falls into the first case of Game 2, so the reductions in Games 6 and 7 still go through; Game 8 involves a statistical argument which does not need to change either.

Games 9–10: These two games involve statistical arguments only, and they have nothing to do with whether the games need to check P''s H_{tag} queries.

Game 11: This game hop goes through for essentially the same reason as in Section 4.2. The difference is that now Game 10 and Game 11 differ if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query, but $\mathcal{A}$ *or* P' makes an $H_{\mathsf{tag}}(pk, K^*)$ query resulting in τ^*. However, pk is freshly generated by P, independent of *both* $\mathcal{A}$ and P''s views. Thus, the distinguishing advantage between Game 10 and Game 11 is still upper-bounded by $H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen})$.

Game 12: This game should read as follows now: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c \wedge \tau^* \neq \tau'$, do what's in the $\mathsf{pw} \neq \mathsf{pw}' \vee ((s^*, T^*) = (s, T) \wedge c^* \neq c)$ case. That is, first find the $\langle \mathsf{pw}, \star, \star \rangle$ record (as per Game 11), then find $\mathcal{A}$'s $H_{\mathsf{tag}}(pk, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \bot$. If there is a unique such query, do the following:

- If $K^* = K$, set $SK := H_{\mathsf{key}}(K)$.
- If $K^* \neq K$, set $SK := \bot$.

In Game 11 SK is outright set to $\bot$, whereas here it checks $\mathcal{A}$'s H_{tag} queries *but does not check P''s H_{tag} query*. Therefore, this game hop has nothing to do with what's different in the new Game 2, and the previous argument still goes through.

Game 13: This game, as well as its argument, need to change quite a bit. We first include a full description of the game.

In the case where $\mathsf{pw} = \mathsf{pw}' \vee (s^*, T^*) = (s, T) \vee (c^*, \tau^*) \neq (c, \tau')$ yet $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge (c^*, \tau^*) = (c, \tau'))$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(pk^*, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \bot$. Otherwise there is a unique such $H_{\mathsf{tag}}(pk^*, K^*)$ query, and

- If there is no or more than one $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, set $SK := \bot$.
- If there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, but $\mathsf{pw}^* \neq \mathsf{pw}$, also set $SK := \bot$.
- If there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, and $\mathsf{pw}^* = \mathsf{pw}$, compute $K := \mathsf{Decap}(sk^*, c^*)$. If $K^* = K$ then set $SK := H_{\mathsf{key}}(K)$, otherwise set $SK := \bot$.

Furthermore, in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, do the same as above, except that we now find $\mathcal{A}$ *or* P''s $H_{\mathsf{tag}}(pk^*, K^*)$ query resulting in τ^*.

We first upper-bound the probability that there is more than one $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record for the same pk^*. Note that in each such record, $(pk^*, \star) \leftarrow \mathsf{Gen}$ is freshly generated, so the probability that there is a collision in pk^* is upper-bounded by $\binom{q_2}{2} \cdot H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen})$.

Next, suppose there is a unique $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record, but $\mathsf{pw}^* \neq \mathsf{pw}$. If there is no $\langle \mathsf{pw}, pk, sk \rangle$ record, then both Game 12 and Game 13 set $SK := \perp$. If there is a $\langle \mathsf{pw}, pk, sk \rangle$ record, then $(pk, \star) \leftarrow \mathsf{Gen}$ is freshly generated, so $\Pr[pk = pk^*] \leq q_2 \cdot H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen})$; and if $pk \neq pk^*$, then $\mathcal{A}$ does not make an $H_{\mathsf{tag}}(\mathsf{pw}, pk)$ query resulting in τ^* (because there is only one H_{tag} query resulting in τ^*, and it is on pk^*), so again both Game 12 and Game 13 set $SK := \perp$.

Assuming neither of the above happens, the previous statistical argument does not really change.

Games 14–16: The statistical arguments for these three games do not need to change.

New game 1: Now that $(\mathsf{pw}')^*$ — the password guess extracted from (s^*, T^*) — is defined, we can finally add (part of) what's in step 2 of Figure 12. Concretely, we change the game as follows:

In the case where $(\mathsf{pw}')^* = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, there must be a pk' defined on the P' side. If there is a unique record $\langle \mathsf{pw}^*, pk', \star \rangle$, and $\mathsf{pw}^* = \mathsf{pw}$, set $SK := H_{\mathsf{key}}(K')$. (In some sense this is the most crucial new game, as it covers the scenario in Figure 11 — which is why we need to make all these changes to begin with. It also corresponds to step 2(c)(ii) in Figure 12. We will deal with step 2(c)(i) later.)

We first argue $\mathsf{pw} = \mathsf{pw}'$ with negligible probability.

Since $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, Game 16 finds $\mathcal{A}$ or P''s H_{tag} query resulting in τ^* (see the description of Game 13). P' makes a singe query to H_{tag}, on input (pk', K'); and the query result is $\tau' = \tau^*$. Therefore, Game 16 finds this query, and does the following (again see the description of Game 13):

1. Find the (unique) record $\langle \mathsf{pw}^*, pk', sk' \rangle$.
2. Check if $\mathsf{pw}^* = \mathsf{pw}$, which is indeed the case.
3. Compute $K := \mathsf{Decap}(sk', c^*)$ and check if $K = K'$. We argue that the check must pass. Steps 1 and 2 above mean that there is a (unique) $\langle \mathsf{pw}, pk', sk' \rangle$ record. But then

$$(c, K') \leftarrow \mathsf{Encap}(pk') \text{ on the } P' \text{ side,}$$
$$c^* = c,$$
$$K = \mathsf{Decap}(sk', c^*) \text{ on the } P \text{ side,}$$

and sk' is the KEM secret key corresponding to pk' (again, because there is a $\langle \mathsf{pw}, pk', sk' \rangle$ record). So $K = K'$ by the correctness property of the KEM scheme.

4. Set $SK := H_{\mathsf{key}}(K)(= H_{\mathsf{key}}(K'))$.

This entire process in Game 16 eventually results in setting $SK := H_{\mathsf{key}}(K')$, which exactly matches the new game. Thus, there is no difference. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{16,\text{new}1} = 0.$$

New game 2: In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge c^* = c$, we find $\mathcal{A}$ *or* P''s $H_{\mathsf{tag}}(pk^*, K^*)$ query resulting in τ^* (except that in new game 1's condition, new game 1 already shortcuts it to $SK := H_{\mathsf{key}}(K')$ without trying to find any H_{tag} query). In this game, we do not check P''s H_{tag} query anymore, and only check $\mathcal{A}$'s H_{tag} query.

There are the following cases:

- If $\tau^* \neq \tau'$, then of course P''s H_{tag} query does not result in τ^* (it results in τ'), so we can safely drop P''s H_{tag} query.
- If $(\mathsf{pw}')^* \neq \mathsf{pw}' \wedge \tau^* = \tau'$, then P''s $H_{\mathsf{tag}}(pk', K')$ query results in τ^*, and we need to argue that this query can be dropped from the check.
  - If $\mathcal{A}$ queries $H_{\mathsf{tag}}(pk', K')$, then new game 2 still finds this query, so new game 1 and new game 2 are identical.
  - If $\mathcal{A}$ does not query $H_{\mathsf{tag}}(pk', K')$, then new game 2 sets $SK := \perp$. We argue that new game 1 also sets $SK := \perp$ except with negligible probability. New game 1 finds P''s $H_{\mathsf{tag}}(pk', K')$ query, then finds the $\langle \mathsf{pw}^*, pk', sk' \rangle$ record, and sets $SK := \perp$ unless $\mathsf{pw}^* = \mathsf{pw} \wedge \mathsf{Decap}(sk', c^*) = K'$. We claim that $\mathsf{pw}^* = \mathsf{pw}$ with negligible probability; in other words, suppose there is a record $\langle \mathsf{pw}, pk, \star \rangle$, then $pk = pk'$ with negligible probability.

    Note that $(pk, \star) \leftarrow \mathsf{Gen}$ and later $H_1(\mathsf{pw}, r)$ is programmed to T/pk; for pk', we have
    $$pk' = \frac{T^*}{R'} = \frac{T^*}{H_1(\mathsf{pw}', r')}.$$
    R' is independent of pk, since $\mathsf{pw} \neq \mathsf{pw}'$ by the condition of this game and thus $R' = H_1(\mathsf{pw}', r')$ is independent of $H_1(\mathsf{pw}, r)$. Furthermore, R' is also independent of T^*, since T^* is determined *before* the $H_1(\mathsf{pw}', r')$ query:
    * If the $H_1(\mathsf{pw}', r')$ query is made by P', then P' makes this query only after receiving T^*, so T^* is determined before the $H_1(\mathsf{pw}', r')$ query.
    * If the $H_1(\mathsf{pw}', r')$ query is made by $\mathcal{A}$, then $\mathcal{A}$'s $H_2(\mathsf{pw}', T^*)$ query must come before $\mathcal{A}$'s $H_1(\mathsf{pw}', r')$ query (otherwise pw' falls into type (1) in Game 15, so $(\mathsf{pw}')^* = \mathsf{pw}'$ which violates the condition

of this game), and T^* must be determined before $\mathcal{A}$'s $H_2(\mathsf{pw}', T^*)$ query, so T^* is determined before the $H_1(\mathsf{pw}', r')$ query.

Since R' is independent of both pk and R', $pk' = T^*/R'$ is independent of pk no matter what the distribution of T^* is. Therefore, $\Pr[pk = pk']$ is upper-bounded by $1/|\mathbb{G}|$ for a single H_1 query and $(q_1 + 1)/|\mathbb{G}|$ overall.

- If $(\mathsf{pw}')^* = \mathsf{pw}' \wedge \tau^* = \tau'$, but there is no or more than one $\langle \mathsf{pw}^*, pk', \star \rangle$ record, then both new game 1 and new game 2 outright sets $SK := \perp$, regardless of H_{tag} queries by $\mathcal{A}$ or P'. Thus, the two games are identical.
- If $(\mathsf{pw}')^* = \mathsf{pw}' \wedge \tau^* = \tau'$, there is a unique $\langle \mathsf{pw}^*, pk', \star \rangle$ record, but $\mathsf{pw}^* \neq \mathsf{pw}$, then again both new game 1 and new game 2 outright sets $SK := \perp$, regardless of H_{tag} queries by $\mathcal{A}$ or P'. Thus, the two games are identical.

The remaining case is that $(\mathsf{pw}')^* = \mathsf{pw}' \wedge \tau^* = \tau'$, there is a unique $\langle \mathsf{pw}^*, pk', \star \rangle$ record, and $\mathsf{pw}^* = \mathsf{pw}$. This is exactly the condition in new game 1, and new game 1 outright sets $SK := H_{\mathsf{key}}(K')$ without trying to find any H_{tag} query (by $\mathcal{A}$ or P'), so the game change does not affect this case. Overall, we conclude that

$$\mathbf{Dist}_{\mathcal{Z}}^{\text{new1,new2}} \leq \frac{q_1 + 1}{|\mathbb{G}|}.$$

Game 17: Note that after new game 2, we never try to find P''s H_{tag} query anymore: in some cases we try to find $\mathcal{A}$'s H_{tag} query, and in other cases we outright set SK to be a certain value or $\perp$. Therefore, the reduction still goes through (it can simulate the P side honestly).

New game 3: In the case where $(\mathsf{pw}')^* = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, there is a unique record $\langle \mathsf{pw}^*, pk', \star \rangle$, and $\mathsf{pw}^* \neq \mathsf{pw}$, set $SK := \perp$. (This corresponds to step 2(c)(i) in Figure 12.)

In this case Game 17 first tries to find $\mathcal{A}$'s (unique) $H_{\mathsf{tag}}(pk^*, K^*)$ query resulting in τ^*. Since $\tau^* = \tau' = H_{\mathsf{tag}}(pk', K')$, we have $pk^* = pk'$. Then Game 17 tries to find the (unique) $\langle \mathsf{pw}^*, pk', \star \rangle$ record, and checks if $\mathsf{pw}^* = \mathsf{pw}$ (see Game 13). Since $\mathsf{pw}^* \neq \mathsf{pw}$, the check never passes, so both Game 17 and new game 3 set $SK := \perp$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{17,\text{new3}} = 0.$$

Summary. We have made the following essential changes to the games:

- In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge c^* = c$, Game 2 tries to find P''s H_{tag} query resulting in τ^* (in addition to $\mathcal{A}$'s query).
- In the case where $(\mathsf{pw}')^* = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, there is a unique record $\langle \mathsf{pw}^*, pk', \star \rangle$, and $\mathsf{pw}^* = \mathsf{pw}$, new game 1 outright sets $SK := H_{\mathsf{key}}(K')$ (without trying to find any H_{tag} query by $\mathcal{A}$ or P').

- In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge c^* = c$, new game 2 drops the check on P''s H_{tag} query introduced in Game 2 (except in new game 1's condition, where all checks on H_{tag} queries have already been dropped).
- In the case where $(\mathsf{pw}')^* = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$, there is a unique record $\langle \mathsf{pw}^*, pk', \star \rangle$, and $\mathsf{pw}^* \neq \mathsf{pw}$, new game 3 outright sets $SK := \perp$.

Overall, the net changes are that in the case where $(\mathsf{pw}')^* = \mathsf{pw}' \wedge (s^*, T^*) \neq (s, T) \wedge (c^*, \tau^*) = (c, \tau')$ and there is a unique record $\langle \mathsf{pw}^*, pk', \star \rangle$,

- If $(\mathsf{pw}')^* = \mathsf{pw}$, then we set $SK := H_{\mathsf{key}}(K')$.
- If $(\mathsf{pw}')^* \neq \mathsf{pw}$, then we set $SK := \perp$.

This exactly matches step 2 of Figure 12, so new game 3 is identical to the ideal world. This completes the proof.

C Proof of Lemma 1

We need two helper lemmas first.

Lemma 2. *Suppose the UNI-PK and OW properties hold for a KEM scheme. Then for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible:*

$pk \leftarrow \mathcal{PK}$
$(c, K) \leftarrow \mathsf{Encap}(pk)$
$K^* \leftarrow \mathcal{A}(pk, c)$
$\mathcal{A}$ wins if $K^* = K$

(Below we call this property UNI-OW.)

Proof. This is straightforward so we only provide a sketch. We claim that the following two games are indistinguishable:

UNI-OW game:	OW game:
$pk \leftarrow \mathcal{PK}$	$(pk, \star) \leftarrow \mathsf{Gen}$
$(c, K) \leftarrow \mathsf{Encap}(pk)$	$(c, K) \leftarrow \mathsf{Encap}(pk)$
$K^* \leftarrow \mathcal{A}(pk, c)$	$K^* \leftarrow \mathcal{A}(pk, c)$
$\mathcal{A}$ wins if $K^* = K$	$\mathcal{A}$ wins if $K^* = K$

This is due to a reduction $\mathcal{R}_{\text{UNI-PK}}$ to the UNI-PK property of the KEM scheme: $\mathcal{R}_{\text{UNI-PK}}$, on input pk, can run either the UNI-OW game or the OW-game — depending on whether its challenge bit $b = 1$ (i.e., pk is uniformly random) or $b = 0$ (i.e., pk is honestly generated) — and output 1 if $\mathcal{A}$ wins and 0 otherwise. The difference between $\Pr[\mathcal{A}$ wins the UNI-OW game$]$ and

Pr[$\mathcal{A}$ wins the OW game] is equal to $\mathcal{R}_{\text{UNI-PK}}$'s advantage, which must be negligible. Since the OW property holds for the KEM scheme, Pr[$\mathcal{A}$ wins the OW game] is also negligible. So Pr[$\mathcal{A}$ wins the UNI-OW game] must be negligible. $\square$

Lemma 3. *Suppose the UNI-PK and ANO properties hold for a KEM scheme. Then for any PPT adversary $\mathcal{A}$,* $\Pr[b^* = 1]$ *in the following two games are negligibly close:*

$$\boxed{\begin{array}{l} \underline{\text{challenge bit } b = 0\text{:}} \\ pk \leftarrow \mathcal{PK} \\ (c, \star) \leftarrow \mathsf{Encap}(pk) \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}} \quad \boxed{\begin{array}{l} \underline{\text{challenge bit } b = 1\text{:}} \\ pk, pk' \leftarrow \mathcal{PK} \\ (c, \star) \leftarrow \mathsf{Encap}(pk') \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}}$$

(Below we call this property UNI-ANO.)

Proof. The proof goes by a sequence of games, and we again only provide a sketch. The following two games

$$\boxed{\begin{array}{l} \underline{\text{challenge bit } b = 0\text{:}} \\ pk \leftarrow \mathcal{PK} \\ (c, \star) \leftarrow \mathsf{Encap}(pk) \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}} \quad \boxed{\begin{array}{l} \underline{\text{challenge bit } b = 0 \text{ in the ANO game:}} \\ (pk, \star) \leftarrow \mathsf{Gen} \\ (c, \star) \leftarrow \mathsf{Encap}(pk) \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}}$$

are indistinguishable due to the UNI-PK property of the KEM scheme (the reduction is very similar to that in the proof of Lemma 2, except that here the reduction copies $\mathcal{A}$'s output bit instead of making a guess and outputting one of $\mathcal{A}$'s RO queries); the following two games

$$\boxed{\begin{array}{l} \underline{\text{challenge bit } b = 0 \text{ in the ANO game:}} \\ (pk, \star) \leftarrow \mathsf{Gen} \\ (c, \star) \leftarrow \mathsf{Encap}(pk) \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}} \quad \boxed{\begin{array}{l} \underline{\text{challenge bit } b = 1 \text{ in the ANO game:}} \\ (pk, \star) \leftarrow \mathsf{Gen} \\ (pk', \star) \leftarrow \mathsf{Gen} \\ (c, \star) \leftarrow \mathsf{Encap}(pk') \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}}$$

are indistinguishable because this is exactly what the ANO property of the KEM scheme says; finally, the following two games

$$\boxed{\begin{array}{l} \underline{\text{challenge bit } b = 1 \text{ in the ANO game:}} \\ (pk, \star) \leftarrow \mathsf{Gen} \\ (pk', \star) \leftarrow \mathsf{Gen} \\ (c, \star) \leftarrow \mathsf{Encap}(pk') \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}} \quad \boxed{\begin{array}{l} \underline{\text{challenge bit } b = 1\text{:}} \\ pk, pk' \leftarrow \mathcal{PK} \\ (c, \star) \leftarrow \mathsf{Encap}(pk') \\ b^* \leftarrow \mathcal{A}(pk, c) \end{array}}$$

are indistinguishable again due to the UNI-PK property of the KEM scheme. $\square$

Proof of Lemma 1. The proof goes by a sequence of games, and we again only provide a sketch. Suppose $\mathcal{A}$ makes q_1, q_2 queries to ROs H_1, H_2, respectively. We start from the game where the challenge bit $b = 0$. To begin with, we claim that $\Pr[b^* = 1]$ does not change if we simplify the game so that $\mathcal{A}$ cannot choose its index i:

challenge bit $b = 0$:	simplified $b = 0$ game:
$pk_1, \ldots, pk_q \leftarrow \mathcal{PK}$	$pk \leftarrow \mathcal{PK}$
$i \leftarrow \mathcal{A}(pk_1, \ldots, pk_q)$	$(c, \star) \leftarrow \mathsf{Encap}(pk)$
$(c, K) \leftarrow \mathsf{Encap}(pk_i)$	$h_1 := H_1(K); h_2 := H_2(K)$
$h_1 := H_1(K); h_2 := H_2(K)$	$b^* \leftarrow \mathcal{A}(pk, c, h_1, h_2)$
$b^* \leftarrow \mathcal{A}(c, h_1, h_2)$	

Note that no matter which index i $\mathcal{A}$ chooses, once i is fixed $\mathcal{A}$'s view is identical to its view in the simplified game (plus redundant public keys pk_{i^*} for $i^* \neq i$, which are not used anywhere in the game and thus do not affect $\Pr[b^* = 1]$). Therefore, letting

$$\begin{aligned} p_{i^*} &= \Pr[b^* = 1 \mid i = i^* \text{ in the original game}], \\ p &= \Pr[b^* = 1 \text{ in the simplified game}], \end{aligned}$$

we have that for any $i^* \in [q]$,

$$p_{i^*} = p.$$

But then

$$\begin{aligned} \Pr[b^* = 1 \text{ in the original game}] &= \sum_{i^*=1}^{q} p_{i^*} \cdot \Pr[i = i^*] \\ &= \sum_{i^*=1}^{q} p \cdot \Pr[i = i^*] \\ &= p \cdot \sum_{i^*=1}^{q} \Pr[i = i^*] = p, \end{aligned}$$

so $Pr[b^* = 1]$ in the original game and the simplified game are equal. (Note that we do not need a hybrid argument here.)

Next, we sample h_1, h_2 at random rather than set them to the real RO outputs:

simplified $b = 0$ game:	intermediate game:
$pk \leftarrow \mathcal{PK}$	$pk \leftarrow \mathcal{PK}$
$(c, \star) \leftarrow \mathsf{Encap}(pk)$	$(c, K) \leftarrow \mathsf{Encap}(pk)$
$h_1 := H_1(K); h_2 := H_2(K)$	$h_1 \leftarrow \{0,1\}^n; h_2 \leftarrow \{0,1\}^{2n}$
$b^* \leftarrow \mathcal{A}(pk, c, h_1, h_2)$	$b^* \leftarrow \mathcal{A}(pk, c, h_1, h_2)$

The two games are identical unless $\mathcal{A}$ queries either $H_1(K)$ or $H_2(K)$, which happens with negligible probability due to a reduction $\mathcal{R}_{\text{UNI-OW}}$ to the UNI-OW property of the KEM scheme (Lemma 2). $\mathcal{R}_{\text{UNI-OW}}$, on input (pk, c), samples $i \leftarrow [q_1 + q_2]$ as a guess that $\mathcal{A}$'s i-th H_1 or H_2 query is on K. Then $\mathcal{R}_{\text{UNI-OW}}$ samples $pk \leftarrow \mathcal{PK}, h_1 \leftarrow \{0,1\}^n, h_2 \leftarrow \{0,1\}^{2n}$ and invokes $\mathcal{A}$ on input (pk, c, h_1, h_2) to $\mathcal{A}$. On $\mathcal{A}$'s i-th H_1 or H_2 query (say the query input is K^*), $\mathcal{R}_{\text{UNI-OW}}$ outputs K^*. Assuming $\mathcal{A}$ queries either $H_1(K)$ or $H_2(K)$, and $\mathcal{R}_{\text{UNI-OW}}$'s guess of index i is correct, $\mathcal{R}_{\text{UNI-OW}}$ simulates the two games perfectly until the "bad event" happens and wins the UNI-OW game. So the above two games are indistinguishable (with a $q_1 + q_2$ reduction loss).

After that, we change the generation of c from using pk_i to using a fresh pk':

intermediate game:	simplified $b = 1$ game:
$pk \leftarrow \mathcal{PK}$	$pk, pk' \leftarrow \mathcal{PK}$
$(c, \star) \leftarrow \mathsf{Encap}(pk)$	$(c, \star) \leftarrow \mathsf{Encap}(pk')$
$h_1 \leftarrow \{0,1\}^n; h_2 \leftarrow \{0,1\}^{2n}$	$h_1 \leftarrow \{0,1\}^n; h_2 \leftarrow \{0,1\}^{2n}$
$b^* \leftarrow \mathcal{A}(pk, c, h_1, h_2)$	$b^* \leftarrow \mathcal{A}(pk, c, h_1, h_2)$

The two games are indistinguishable, due to the UNI-ANO property of the KEM scheme (Lemma 3). (The only difference is that here $\mathcal{A}$ additionally sees h_1, h_2, which are not used anywhere in the game.)

Finally, we bring back $\mathcal{A}$'s ability to choose an index i and arrive at the game where the challenge bit $b = 1$:

simplified $b = 1$ game:	challenge bit $b = 1$:
$pk, pk' \leftarrow \mathcal{PK}$	$pk', pk_1, \ldots, pk_q \leftarrow \mathcal{PK}$
$(c, \star) \leftarrow \mathsf{Encap}(pk')$	$i \leftarrow \mathcal{A}(pk_1, \ldots, pk_q)$
$h_1 \leftarrow \{0,1\}^n; h_2 \leftarrow \{0,1\}^{2n}$	$(c, \star) \leftarrow \mathsf{Encap}(pk')$
$b^* \leftarrow \mathcal{A}(pk, c, h_1, h_2)$	$h_1 \leftarrow \{0,1\}^n; h_2 \leftarrow \{0,1\}^{2n}$
	$b^* \leftarrow \mathcal{A}(c, h_1, h_2)$

An argument similar to the first game hop shows that $\Pr[b^* = 1]$ in the two games above are equal, completing the proof. $\square$